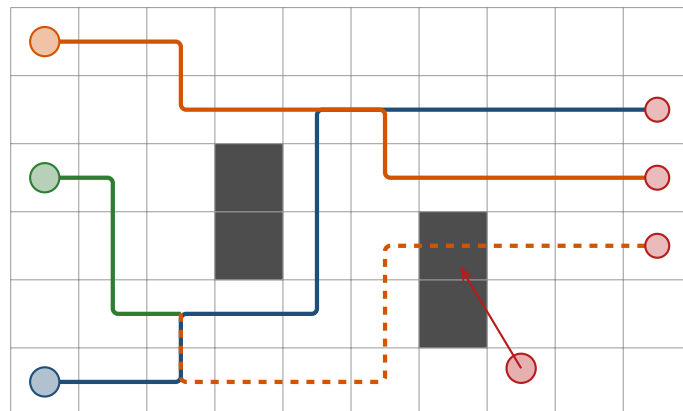


Multi-Agent Path Planning and Drone Collision Avoidance

A Textbook of Algorithms:
From Graph Search to Hybrid Swarm Systems



The Searchbook Project

September 9, 2026

Multi-Agent Path Planning and Drone Collision Avoidance

A Textbook of Algorithms: From Graph Search to Hybrid Swarm Systems

Typeset with L^AT_EX (pdfT_EX), TikZ and pgfplots. Figures were drawn in TikZ or generated from the Python reference implementations that accompany the book.

This book was written to accompany a twelve-week research training plan on multi-agent path planning and collision avoidance for drone swarms. It is intended for self-study. All algorithms described here are established published methods; the original sources are cited in each chapter and collected in the bibliography.

Source code, figures and L^AT_EX sources are distributed together with this book so that every example can be reproduced and every figure regenerated.

Compiled on September 9, 2026.

Contents

Preface	iv
Notation	vii
I Foundations	1
II Single-Agent Graph Search	2
III Multi-Agent Path Finding	3
IV Local and Reactive Collision Avoidance	4
V Sampling-Based Motion Planning	5
VI Tracking and Prediction	6
VII Optimization and Control	7
VIII Putting It Together	8
IX Appendices	9
A Mathematical Refresher	10
A.1 Vectors and matrices	10
A.2 Calculus for motion	14
A.3 Probability and Gaussians	16
A.4 Optimization	20
A.5 Graphs and the Laplacian	25
A.6 Complexity and data structures	27
A.7 Summary	28
Bibliography	30
Index	31

List of Algorithms

Preface

Why this book exists

Imagine a small fleet of drones that must fly through the same airspace at the same time. Each drone has a job to do and a route to fly. Before take-off, a planner must make sure that no two routes collide. During the flight, something unplanned happens: another drone, not part of the fleet and not talking to it, crosses the formation. The fleet must notice it, guess where it is going, get out of its way, and then return to the plan—or, if that is impossible, make a new plan and check it against the rest of the fleet.

Every part of that story is a well-studied algorithmic problem. Planning a route is *graph search*. Planning routes for many drones at once without collisions is *multi-agent path finding*. Reacting to a surprise in a fraction of a second is *local collision avoidance*. Guessing where the intruder is going is *tracking and trajectory prediction*. Making a new plan without starting from scratch is *incremental replanning*. Keeping the fleet in formation and within radio range while doing all of this is *control*. The individual algorithms are established and published. What is not settled is how to combine them into one system that keeps the guarantees of the global plan, reacts fast enough to be safe, and can be evaluated honestly.

This book teaches all of those algorithms, one at a time, from first principles, and then shows how they fit together into a hybrid swarm planner. It was written to accompany a twelve-week research training plan; the plan is reproduced, with week-by-week reading and coding assignments, in ??.

Who should read it

The book is for a graduate student or engineer who is starting research on drone swarms, or on multi-robot motion planning more generally, and who wants to *implement* the algorithms rather than merely recognise their names. You should be comfortable with Python and with the mathematics of a first course in linear algebra, calculus and probability. Everything else is introduced when it is needed, and [Appendix A](#) collects the mathematical facts that the book relies on. No prior knowledge of robotics or planning is assumed.

What you will be able to do

When you have worked through the book and its exercises you will be able to

- implement and explain A^* and space-time A^* , and say exactly why they return optimal paths under the required conditions;
- repair a path with D^* Lite instead of replanning from scratch, and measure when that is worth it;
- represent and detect the conflicts of a multi-agent plan, and solve the problem optimally with conflict-based search or approximately, with a guaranteed bound, with ECBS;
- explain velocity obstacles, reciprocal velocity obstacles and ORCA using relative position and velocity, and use them to keep drones apart;

- track an unknown drone from noisy observations with a Kalman filter, and compare a constant-velocity predictor with a learned one;
- trigger local avoidance from a time-to-collision or safety-horizon rule, return to the global route, or replan;
- enforce formation and communication-range constraints, and
- evaluate a hybrid planner with reproducible metrics and controlled scenarios.

How the book is organised

Part I, Foundations.

?? states the problem and previews the hybrid architecture that the whole book builds towards. ?? is a toolbox: graphs, grids, configuration space, time-indexed paths, simple motion models, the geometry of moving discs, Gaussians and priority queues.

Part II, Single-Agent Graph Search.

Dijkstra's algorithm (??), A^* with admissible and consistent heuristics and its space-time version (??), incremental replanning with LPA^* and D^* Lite (??) and anytime search with ARA^* (??).

Part III, Multi-Agent Path Finding.

The problem and its conflicts (??), prioritized planning (??), conflict-based search (??), bounded-suboptimal search with ECBS (??), and the coupled and constructive alternatives M^* , Push-and-Swap and Push-and-Rotate (??).

Part IV, Local and Reactive Collision Avoidance.

Velocity obstacles (??), reciprocal avoidance with RVO and ORCA (??), the dynamic window approach (??) and artificial potential fields (??).

Part V, Sampling-Based Motion Planning.

RRT (??) and the asymptotically optimal RRT^* with informed sampling (??).

Part VI, Tracking and Prediction.

The Kalman filter (??), its nonlinear relatives and the particle filter (??), and trajectory prediction from constant-velocity baselines to LSTMs and Transformers (??).

Part VII, Optimization and Control.

Model predictive control (??), mixed-integer linear programming (??) and consensus and formation control (??).

Part VIII, Putting It Together.

The hybrid collision-avoidance architecture (??) and how to evaluate it (??).

How to read it

Read the chapters in order if you can: each part builds on the previous ones, and the notation is introduced once. If you are short of time, the training plan's reading order is a good fast path: ?? (A^*), ?? (D^* Lite), ?? (CBS), ?? (the MAPF problem), ???? (velocity obstacles and ORCA), ?? (DWA), ???? (RRT and RRT^*), ?? (MPC) and ?? (trajectory prediction), then ??. Every chapter ends with a box that says where its algorithm sits in the hybrid system, so you can also read by *layer*: the global planner (Parts II and III), the local safety layer (Part IV), the prediction layer (Part VI) and the constraints (Part VII).

Conventions

Every chapter opens with a box of *learning objectives* and closes with a *summary* box and exercises. Inside a chapter you will meet five kinds of coloured boxes: *key idea* (green) states the one thing to remember; *in the drone system* (purple) says how the algorithm is used in the hybrid architecture; *pitfall* (orange) warns about a mistake that is easy to make; *historical note* (grey) gives context; and plain notes carry their own titles. Definitions, examples, theorems and remarks share one counter per chapter, so “Example 4.3” comes after “Definition 4.2”. Exercises are marked with one to three stars (★ routine, ★★ needs thought, ★★★ a small project), and hints or solutions for a selection of them are in ???. Where the training plan assigns a priority to an algorithm it is shown as a tag such as **Essential**. Pseudocode is written in a uniform style with numbered lines; the text refers to line numbers. Vectors are bold ($\mathbf{p}$, $\mathbf{v}$), matrices are bold capitals ($\mathbf{A}$), and the full notation is listed in the table that follows this preface.

The code

Every algorithm in the book comes with a Python implementation in the `code/` folder of the L^AT_EX project. The implementations are deliberately simple—plain Python and NumPy, a few hundred lines each, no framework—because the goal is that you can read them in one sitting and then write your own. Each file runs a self-test when executed (`python3 code/ch04_astar.py`), and the numbers quoted in the worked examples were produced by that code. The figures that show algorithm output were generated by the scripts in `code/figures/` from fixed random seeds, so you can regenerate and modify them. The listings in the text are excerpts; the files contain the complete programs.

Sources

None of the algorithms in this book is new. Each chapter cites the paper in which its algorithm was introduced and a textbook that treats it at greater length, and the *further reading* paragraph at the end of every chapter points to the developments that the chapter leaves out. The bibliography lists all sources. Where the book simplifies an algorithm for teaching, it says so and names the full version.

Errata

The manuscript was reviewed in several rounds for technical accuracy, completeness against the training plan, and clarity. Mistakes will nevertheless remain; if you find one, the L^AT_EX sources and code are distributed with the book precisely so that it can be corrected.

Notation

This book draws on eight fields that grew up apart, and their alphabets collide. The tables below list every symbol used in more than one place, say what it means, and name the chapter that introduces it. Where a letter really does carry two meanings—**Q** and **R** are the clearest case—the table says so rather than pretend otherwise.

Conventions

Fonts. Scalars are italic (v, t, c, λ), vectors are bold lower case (**p**, **v**, **x**, **u**, **z**) and matrices are bold upper case (**A**, **P**, **Σ**). The zero vector is **0**, the identity matrix **I** and the vector of ones **1**; their sizes follow from the context. Sets and spaces are calligraphic ($\mathcal{C}$, $\mathcal{C}_{\text{free}}$, $\mathcal{C}_i$, $\mathcal{W}$, $\mathcal{N}$), with the one classical exception that the vertex and edge sets of a graph keep the plain letters V and E ; data structures are small capitals (OPEN, CLOSED, FOCAL) and names taken from code are typeset as `this`. $\mathbf{A}^\top$ is the transpose and $\mathbf{A}^{-1}$ the inverse; $\|\cdot\|$ is the Euclidean norm unless a subscript says otherwise, and $|\cdot|$ is an absolute value or, for a finite set, its number of elements.

Indices. A subscript k or t is a time index: $\mathbf{x}_k$ is the state at step k and $\pi[t]$ the vertex occupied at step t . A subscript i or j is an agent index: π_i is the path of agent a_i , $\mathbf{p}_i$ its position, r_i its radius. When both are needed the agent moves up and the time stays down, as in $\mathbf{p}_k^i$; such a superscript is a label, never an exponent, while $\mathbf{A}^k$ is a power. Parenthesised superscripts count samples: $\mathbf{x}^{(i)}$ is the i -th particle or sigma point. Search chapters count time in unit steps, $t = 0, 1, 2, \dots$; control and estimation chapters count steps of the sampling interval, so step k is the instant $k \Delta t$.

Decorations. A star marks an optimal quantity (g^* , h^* , C^* , J^*), a hat an estimate from a filter or a predictor ($\hat{\mathbf{x}}_k$, $\hat{\mathbf{p}}_{T+h}$), a bar an average ($\bar{x}$), and a superscript minus a *prediction* made before the current measurement has been used ($\hat{\mathbf{x}}_k^-$, $\mathbf{P}_k^-$).

General mathematics

Symbol	Meaning	Introduced in
$\mathbb{R}, \mathbb{N}, \mathbb{Z}$	reals, naturals including 0, integers; $\mathbb{R}^n$ holds the n -vectors	??
$\mathbf{x}, \mathbf{A}$	vector (bold lower case), matrix (bold upper case)	??
0 , 1 , I	zero vector, vector of ones, identity matrix	??
$\mathbf{A}^\top, \mathbf{A}^{-1}$	transpose and inverse of a matrix	??
$\ \mathbf{x}\ , a $	Euclidean norm; absolute value, or size of a finite set	??
$\mathbf{a} \cdot \mathbf{b}, \mathbf{a} \times \mathbf{b}$	dot and cross product; $\mathbf{a}^\perp = (-a_y, a_x)$ is a quarter turn	??
$A \oplus B$	Minkowski sum; inflating obstacles by the drone radius	??
$\arg \min, \arg \max$	the argument that minimises or maximises a function	??
$\text{diag}(\cdot), \text{tr}(\cdot)$	diagonal matrix built from a list; trace of a matrix	??
$\mathcal{O}(f(n))$	asymptotic upper bound in the size of the input	??
$\hat{x}, \bar{x}, x^*, x^-$	estimate, average, optimal value, prediction before the update	??

Graphs, grids and search

Symbol	Meaning	Introduced in
$G = (V, E)$	graph with vertex set V and edge set E ; G_4 , G_8 are grids	??
$c(u, v)$	cost of the edge (u, v) , also written w ; costs are non-negative	??
$\text{Succ}(u)$, $\text{Pred}(u)$	successors and predecessors; b is the branching factor	??
s , g	start and goal vertex; the goal is t in ?? and γ from ?? on	??
π , $\text{cost}(\pi)$, $L(\pi)$	a path, its cost, and its Euclidean length	??
$\text{dist}(u, v)$	shortest-path distance; written $\delta(s, v)$ in ??	??
(v, t) , T	space-time state (vertex v at step t); planning horizon	??
$g(n)$, $g^*(n)$	best cost from s to node n found so far; its optimum	??
$h(n)$, $h^*(n)$	heuristic estimate of the cost to go; the true cost to go	??
$f(n)$, C^*	priority $g + h$; optimal solution cost $C^* = h^*(s)$	??
OPEN, CLOSED	the frontier (a priority queue) and the expanded nodes	??
FOCAL, $f_{\min}$	focal band $\{n \in \text{OPEN} : f(n) \leq w f_{\min}\}$; $f_{\min}$ bounds C^*	??
$w \geq 1$	suboptimality factor of weighted A^* , focal search and ECBS	??
ε	inflation factor of ARA^* ; $w = 1 + \varepsilon$ in A^*_ε	??
$\text{rhs}(s)$	one-step look-ahead value; $g(s) = \text{rhs}(s)$ is local consistency	??
$\text{key}(s)$, k_m	two-component priority and key modifier of D^* Lite	??

Multi-agent path finding

Symbol	Meaning	Introduced in
k , a_i	number of agents; agent i , with start s_i and goal g_i or γ_i	??
π_i , $\pi_i[t]$	path of a_i and the vertex it occupies at step t	??
$\text{cost}(\pi_i)$, T_i	time of final arrival at the goal; last stored index of π_i	??
$\Pi = (\pi_1, \dots, \pi_k)$	a plan: one time-indexed path per agent	??
$\text{SoC}(\Pi)$	sum of costs: the total of $\text{cost}(\pi_i)$ over all agents	??
$\text{MS}(\Pi)$	makespan: the largest $\text{cost}(\pi_i)$, $\text{MS} \leq \text{SoC} \leq k \text{MS}$	??
$\langle a_i, a_j, v, t \rangle$	vertex conflict: both agents occupy v at time t	??
$\langle a_i, a_j, u, v, t \rangle$	edge or swapping conflict on $\{u, v\}$ between t and $t + 1$	??
$\langle a_i, v, t \rangle$	vertex constraint: a_i may not occupy v at time t	??
$\mathcal{C}_i$, N	constraints on a_i ; constraint-tree node $N.C$, $N.\pi$, $N.\text{cost}$	??
σ	priority order: a permutation of the agents, $\sigma(1)$ first	??
h_c	number of remaining conflicts, the FOCAL heuristic of ECBS	??
$C(v)$, $\Psi(v, v')$	collision set of a joint vertex and of a joint move; policy ϕ_i	??

Geometry and motion

Symbol	Meaning	Introduced in
$\mathcal{W}, \mathcal{O}$	workspace and the obstacle region inside it	??
$\mathcal{C}, \mathcal{C}_{\text{free}}, \mathcal{C}_{\text{obs}}$	configuration space, its free part, its obstacle region	??
$D(\mathbf{c}, r)$	closed disc (in 3D, ball) of radius r around $\mathbf{c}$	??
$r_i, R = r_A + r_B$	radius of an agent; collision distance of a pair	??
$\mathbf{p}, \mathbf{v}, \mathbf{a}$	position, velocity, acceleration; limits v_{max} and a_{max}	??
Δt	time step (sampling interval), in seconds	??
$\mathbf{x}, \mathbf{u}$	state and control input of a motion model	??
$\mathbf{A}, \mathbf{B}$	state-transition and input matrix of the double integrator	??
$d_{ij}(t), d_{\min}$	separation of two agents; its minimum over the execution	??
$d_{\text{stop}}(v)$	stopping distance $v^2/(2a_{\text{max}})$ of a braking drone	??
t^*, t_c	time of closest approach and time to collision of $\mathbf{p}_{\text{rel}}, \mathbf{v}_{\text{rel}}$	??
τ	horizon of the reactive layer; a time to collision in ??	??
$VO_{A B}, VO_{A B}^\tau$	velocity obstacle and its truncation at τ ; $CC_{A B}$ is its cone	??
$RVO_{A B}$	reciprocal velocity obstacle, apex at $\frac{1}{2}(\mathbf{v}_A + \mathbf{v}_B)$	??
$ORCA_{A B}^\tau$	the ORCA half-plane of velocities A may choose	??
$\mathbf{u}, \mathbf{n}$	smallest change of the relative velocity and the outward normal	??
$\mathbf{v}^{\text{pref}}, \mathbf{v}^{\text{new}}$	preferred velocity and the velocity finally commanded	??
V_s, V_a, V_d, V_r	possible, admissible, dynamic-window and resulting velocities	??
$U(\mathbf{p}), \rho, \rho_0$	potential field; clearance and influence distance	??
$\mathcal{X}_{\text{free}}, \mathcal{X}_{\text{obs}}$	free and blocked parts of the state space, the $\mathcal{C}_{\text{free}}, \mathcal{C}_{\text{obs}}$ above	??
$T = (V, E), \eta$	tree grown by RRT and its step size; Nearest, Steer extend it	??

Probability and filtering

Symbol	Meaning	Introduced in
$\mathbb{P}(\cdot), p(\mathbf{x})$	probability of an event; density of a random vector	??
$\mathbb{E}[\cdot], \text{Var}, \text{Cov}$	expectation, variance, covariance	??
$\mathcal{N}(\mu, \Sigma)$	Gaussian of mean μ , covariance Σ ; σ_i, ρ_{ij} are its spreads	??
$\mathcal{E}_k$	$k\sigma$ covariance ellipse: Mahalanobis distance k from the mean	??
$\mathbf{x}_k, \mathbf{z}_k$	state and measurement at step k ; $\mathbf{z}_{1:k}$ all up to k	??
$\hat{\mathbf{x}}_k^-, \mathbf{P}_k^-$	predicted mean and covariance, before $\mathbf{z}_k$ is used	??
$\hat{\mathbf{x}}_k, \mathbf{P}_k$	updated (posterior) mean and covariance	??
$\mathbf{F}_k$	state-transition matrix, or the Jacobian $\partial f / \partial \mathbf{x}$	??
$\mathbf{H}_k$	measurement matrix, or the Jacobian $\partial h / \partial \mathbf{x}$	??
$\mathbf{Q}_k, \mathbf{R}_k$	process-noise and measurement-noise covariance	??
$\mathbf{w}_k, \mathbf{v}_k$	process noise and measurement noise, of covariance $\mathbf{Q}_k, \mathbf{R}_k$	??
$\mathbf{y}_k, \mathbf{S}_k$	innovation $\mathbf{z}_k - h(\hat{\mathbf{x}}_k^-)$ and its covariance	??
$\mathbf{K}_k$	Kalman gain $\mathbf{P}_k^- \mathbf{H}_k^T \mathbf{S}_k^{-1}$	??
$\mathcal{X}_i, \mathbf{x}^{(i)}, w^{(i)}$	sigma point; particle i and its weight	??
$T_{\text{obs}}, T_{\text{pred}}$	observation window and prediction horizon; h is a horizon step	??
ADE, FDE	average and final displacement error; minADE_K over K samples	??
$\mathbf{Q}, \mathbf{K}, \mathbf{V}$	query, key and value matrices of attention; this meaning only here	??

Two warnings about letters. $\mathbf{Q}$ and $\mathbf{R}$ are *noise covariances* in Part VI, where they say how far the motion model and the sensor may be trusted, and *cost weights* in Part VII, where they say how much a tracking error and a control effort cost. The two roles are unrelated, they carry different units, and no equation in this book uses both at once; ?? repeats the warning where the weights appear. Second, this book follows the estimation literature and writes $\mathbf{F}$ for the state-transition matrix and $\mathbf{H}$ for the measurement matrix of a filter, where control

texts usually write $\mathbf{A}$ and $\mathbf{C}$. The plant models of ?? and the controller of ?? do use $\mathbf{A}$ and $\mathbf{B}$, because there they describe a vehicle and not a filter.

Optimization and control

Symbol	Meaning	Introduced in
J, J^*	objective of an optimization problem and its optimal value	??
subject to	introduces the constraints of an optimization problem	??
N	prediction horizon in steps of Δt ; $k = 0$ is the present	??
$\mathbf{x}_k^{\text{ref}}$	reference state at step k of the horizon	??
$\mathbf{Q} \succeq \mathbf{0}, \mathbf{R} \succ \mathbf{0}$	weights on the state error and on the control effort	??
$\mathbf{P} \succeq \mathbf{0}$	terminal cost weight on the last predicted state	??
$\mathbf{S}_x, \mathbf{S}_u$	prediction matrices that stack the horizon into one map	??
$\mathbf{z}$	stacked decision variable of the resulting QP or LP	??
$\mathbf{H}, \mathbf{f}, \mathbf{G}$	Hessian, linear term and constraint matrix, $\mathbf{l} \leq \mathbf{G}\mathbf{z} \leq \mathbf{u}$	??
$\mathbf{y}, \rho$	Lagrange multipliers and penalty parameter of the ADMM solver	??
s_k	slack variable that softens a constraint at step k	??
ε	tolerance of a numerical solver (residuals, integrality)	??
$\mathbf{c}, \mathbf{A}, \mathbf{b}$	LP data: $\min \mathbf{c}^\top \mathbf{z}$ over $\mathbf{A}\mathbf{z} \leq \mathbf{b}$; P is the feasible polyhedron	??
$z_j \in \mathbb{Z}, b$	integer variable and binary variable of a mixed-integer program	??
M	big- M constant: $b = 1$ switches an inequality off	??

Networks and consensus

Symbol	Meaning	Introduced in
$G = (V, E)$	communication graph: the drones and the pairs that can talk	??
N_i	the neighbours of drone i in that graph	??
$\mathbf{A} = (a_{ij}), \mathbf{D}$	adjacency matrix; degree matrix $\text{diag}(d_1, \dots, d_n)$	??
$\mathbf{L} = \mathbf{D} - \mathbf{A}$	graph Laplacian; symmetric, $\mathbf{L}\mathbf{1} = \mathbf{0}$	??
λ_2, λ_n	algebraic connectivity (Fiedler value); largest eigenvalue of $\mathbf{L}$	??
$\bar{\mathbf{x}}, \delta$	average of the states; disagreement vector $\mathbf{x} - \bar{\mathbf{x}}\mathbf{1}$	??
ε	step size of discrete consensus, $\mathbf{x}_{k+1} = (\mathbf{I} - \varepsilon\mathbf{L})\mathbf{x}_k$	??
R_{comm}	communication range: $\{i, j\} \in E$ iff $\ \mathbf{p}_i - \mathbf{p}_j\ \leq R_{\text{comm}}$	??
$\mathbf{o}_i, \mathbf{d}_{ij}$	formation offset of drone i ; desired displacement $\mathbf{o}_j - \mathbf{o}_i$	??
e_F, e_{max}	formation error (RMS displacement violation) and its tolerance	??
k_p, k_v, k_d	position, velocity and damping gains of second-order consensus	??

Part I

Foundations

Part II

Single-Agent Graph Search

Part III

Multi-Agent Path Finding

Part IV

Local and Reactive Collision Avoidance

Part V

Sampling-Based Motion Planning

Part VI

Tracking and Prediction

Part VII

Optimization and Control

Part VIII

Putting It Together

Part IX

Appendices

Mathematical Refresher

Learning objectives

After working through this appendix you will be able to

- use the matrix identities (transpose, inverse, eigen-decomposition, definiteness, block matrices, Schur complement, least squares) that the filtering, control and consensus chapters rely on;
- linearise a nonlinear map with its Jacobian, bound the error, and step a double integrator forward exactly;
- write down a multivariate Gaussian, transform it affinely, condition it on a measurement, and gate with the Mahalanobis distance;
- tell a convex problem from a non-convex one, read the KKT conditions, and say what linear, quadratic and mixed-integer programs are and how each is solved;
- form the Laplacian of a graph and explain why its second eigenvalue measures connectivity;
- read a complexity bound and choose the right Python data structure for a search.

This appendix collects the mathematics that the chapters use without proving it. It is a refresher, not a course: every result is stated exactly, short proofs are given in full, long ones are sketched with a pointer to a textbook, and each result names the chapters that depend on it. Read a section when a chapter sends you here; nothing has to be read in order. The notation is that of the notation table at the front of the book, and every number quoted in the examples is produced by the script `code/figures/gen_appB_numbers.py`, so you can check your own hand calculations against it.

Key idea

Four moves carry almost every derivation in this book: diagonalise a symmetric matrix ($\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$), linearise a nonlinear map (keep the Jacobian, bound the rest), push a Gaussian through an affine map and condition it on what was measured (the Kalman filter is nothing else), and check convexity before trusting a local minimum.

A.1 Vectors and matrices

A.1.1 Vectors, norms and products

A vector $\mathbf{x} \in \mathbb{R}^n$ is a column of n real numbers $x_1, \dots, x_n$; its transpose $\mathbf{x}^T$ is the corresponding row. Positions, velocities and filter states are vectors of this kind, with $n = 2$ or 3 for a point

in the plane or in space and a larger n for a stacked state such as $\mathbf{x} = (\mathbf{p}, \mathbf{v})$.

Definition A.1 (Norms, dot product, cross product). For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ the **dot product** is $\mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top \mathbf{y} = \sum_i x_i y_i$. The **Euclidean norm** is $\|\mathbf{x}\| = \sqrt{\mathbf{x}^\top \mathbf{x}}$, the **1-norm** is $\|\mathbf{x}\|_1 = \sum_i |x_i|$ and the **∞ -norm** is $\|\mathbf{x}\|_\infty = \max_i |x_i|$. Every norm satisfies $\|\mathbf{x}\| \geq 0$ with equality only for $\mathbf{x} = \mathbf{0}$, $\|\alpha \mathbf{x}\| = |\alpha| \|\mathbf{x}\|$, and the triangle inequality $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$. For $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$ the **cross product** is

$$\mathbf{a} \times \mathbf{b} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)^\top, \quad (\text{A.1})$$

and for $\mathbf{a}, \mathbf{b} \in \mathbb{R}^2$ the scalar $\mathbf{a} \times \mathbf{b} = a_x b_y - a_y b_x = \mathbf{a}^\perp \cdot \mathbf{b}$, with $\mathbf{a}^\perp = (-a_y, a_x)^\top$ the quarter turn of $\mathbf{a}$.

Three facts about these products are used all over the geometry of ??????. First, $\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$ with θ the angle between the vectors, so $|\mathbf{x} \cdot \mathbf{y}| \leq \|\mathbf{x}\| \|\mathbf{y}\|$ (the **Cauchy–Schwarz inequality**) and two vectors are orthogonal exactly when their dot product is zero. Second, the **projection** of $\mathbf{x}$ onto a unit vector $\mathbf{u}$ is $(\mathbf{x} \cdot \mathbf{u}) \mathbf{u}$, which is how the distance from a point to a segment is computed in ??. Third, $\mathbf{a} \times \mathbf{b}$ in $\mathbb{R}^3$ is orthogonal to both factors with length $\|\mathbf{a}\| \|\mathbf{b}\| \sin \theta$, and the planar scalar $\mathbf{a} \times \mathbf{b}$ is positive exactly when $\mathbf{b}$ lies counter-clockwise from $\mathbf{a}$: this sign is the left-or-right test behind the half-plane and cone tests of ?????. The grid heuristics of ?? are norms of the displacement $\mathbf{d}$ to the goal: Manhattan $\|\mathbf{d}\|_1$, Euclidean $\|\mathbf{d}\|$, Chebyshev $\|\mathbf{d}\|_\infty$.

A.1.2 Matrices, transpose, inverse and rank

A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ maps $\mathbb{R}^n$ to $\mathbb{R}^m$ by $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$; the product $\mathbf{A}\mathbf{B}$, with entries $(\mathbf{A}\mathbf{B})_{ij} = \sum_k a_{ik} b_{kj}$, is the composition “first $\mathbf{B}$, then $\mathbf{A}$ ” and is not commutative in general. The **transpose** $\mathbf{A}^\top$ swaps rows and columns (a square $\mathbf{A}$ with $\mathbf{A}^\top = \mathbf{A}$ is **symmetric**); the **inverse** $\mathbf{A}^{-1}$ of a square matrix satisfies $\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$, when it exists; the **rank** is the number of linearly independent columns, which equals the number of linearly independent rows; the **trace** $\text{tr } \mathbf{A} = \sum_i a_{ii}$ and the **determinant** $\det \mathbf{A}$ are defined for square matrices; and an **orthogonal** matrix, $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$, such as a rotation, preserves norms and dot products. The following equivalences are the working definition of invertibility.

Proposition A.2 (Invertibility). *For a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ the following are equivalent: $\mathbf{A}$ is invertible; $\text{rank } \mathbf{A} = n$; $\det \mathbf{A} \neq 0$; $\mathbf{A}\mathbf{x} = \mathbf{0}$ only for $\mathbf{x} = \mathbf{0}$; $\mathbf{A}$ has no zero eigenvalue. In that case $\mathbf{A}\mathbf{x} = \mathbf{b}$ has the unique solution $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$.*

Table A.1 lists the algebra of transposes, inverses, traces and determinants that the derivations of ?????? use without comment; the block-matrix rows are explained in Appendix A.1.4.

A.1.3 Eigenvalues, symmetric matrices and definiteness

Definition A.3 (Eigenvalues and eigenvectors). A scalar λ and a non-zero vector $\mathbf{q}$ with $\mathbf{A}\mathbf{q} = \lambda \mathbf{q}$ are an **eigenvalue** and an **eigenvector** of the square matrix $\mathbf{A}$. The eigenvalues are the roots of $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$.

For a general real matrix the eigenvalues may be complex. The matrices that carry uncertainty, cost and connectivity in this book are symmetric, and for them everything is real and orthogonal.

Theorem A.4 (Spectral theorem). *A symmetric matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ has n real eigenvalues $\lambda_1 \geq \dots \geq \lambda_n$ with orthonormal eigenvectors $\mathbf{q}_1, \dots, \mathbf{q}_n$, so that*

$$\mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top = \sum_{i=1}^n \lambda_i \mathbf{q}_i \mathbf{q}_i^\top, \quad \mathbf{Q} = [\mathbf{q}_1 \cdots \mathbf{q}_n], \quad \mathbf{Q}^\top \mathbf{Q} = \mathbf{I}, \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n). \quad (\text{A.2})$$

Table A.1. Matrix identities used in the book. All inverses are assumed to exist; λ_i are the eigenvalues; $\mathbf{S} = \mathbf{A} - \mathbf{B}\mathbf{D}^{-1}\mathbf{C}$ is the Schur complement of $\mathbf{D}$ in the block matrix $\mathbf{M}$ of [Theorem A.9](#).

Identity	Remark
$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$, $(\mathbf{AB})^{-1} = \mathbf{B}^{-1} \mathbf{A}^{-1}$	the order reverses
$(\mathbf{A}^\top)^{-1} = (\mathbf{A}^{-1})^\top$	written $\mathbf{A}^{-\top}$
$\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$, $\text{tr} \mathbf{A} = \sum_i \lambda_i$	the trace is cyclic
$\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}$, $\det \mathbf{A}^\top = \det \mathbf{A}$, $\det \mathbf{A} = \prod_i \lambda_i$	$\det \mathbf{A}^{-1} = 1/\det \mathbf{A}$
$\mathbf{x}^\top \mathbf{A} \mathbf{x} = \frac{1}{2} \mathbf{x}^\top (\mathbf{A} + \mathbf{A}^\top) \mathbf{x}$	only the symmetric part matters
$\mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top$, $\mathbf{A}^{-1} = \mathbf{Q} \mathbf{\Lambda}^{-1} \mathbf{Q}^\top$ ($\mathbf{A}$ symmetric)	Theorem A.4
$\mathbf{A} = \mathbf{L} \mathbf{L}^\top$ ($\mathbf{A} \succ \mathbf{0}$, $\mathbf{L}$ lower triangular)	Cholesky factorisation
$\det \mathbf{M} = \det \mathbf{S} \det \mathbf{D}$, $(\mathbf{M}^{-1})_{11} = \mathbf{S}^{-1}$	block matrices, Theorem A.9
$(\mathbf{A} + \mathbf{UCV})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1} \mathbf{U} (\mathbf{C}^{-1} + \mathbf{VA}^{-1} \mathbf{U})^{-1} \mathbf{VA}^{-1}$	Woodbury identity, Proposition A.10
$\nabla_{\mathbf{x}}(\mathbf{b}^\top \mathbf{x}) = \mathbf{b}$, $\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = (\mathbf{A} + \mathbf{A}^\top) \mathbf{x}$	derivatives, Appendix A.2

The proof is in every linear-algebra text [2, Appendix A]. The decomposition says that a symmetric matrix is a pure scaling by λ_i along each of n orthogonal axes $\mathbf{q}_i$; hence $\mathbf{A}^k = \mathbf{Q} \mathbf{\Lambda}^k \mathbf{Q}^\top$ and, if no eigenvalue vanishes, $\mathbf{A}^{-1} = \mathbf{Q} \mathbf{\Lambda}^{-1} \mathbf{Q}^\top$. The **quadratic form** $\mathbf{x}^\top \mathbf{A} \mathbf{x} = \sum_i \lambda_i (\mathbf{q}_i^\top \mathbf{x})^2$ is a weighted sum of squared coordinates along the axes, and its sign decides whether a covariance is valid, a cost is convex and a consensus protocol converges.

Definition A.5 (Positive definite and semidefinite matrices). A symmetric matrix $\mathbf{A}$ is **positive semidefinite** (PSD), written $\mathbf{A} \succeq \mathbf{0}$, if $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ for all $\mathbf{x}$, and **positive definite** (PD), written $\mathbf{A} \succ \mathbf{0}$, if $\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$. $\mathbf{A} \succeq \mathbf{B}$ means $\mathbf{A} - \mathbf{B} \succeq \mathbf{0}$.

Proposition A.6 (Tests and closure properties). *Let $\mathbf{A}$ be symmetric.*

- (i) $\mathbf{A} \succeq \mathbf{0}$ iff all eigenvalues are ≥ 0 , and $\mathbf{A} \succ \mathbf{0}$ iff all are > 0 . A PD matrix is invertible and its inverse is PD.
- (ii) $\mathbf{A} \succeq \mathbf{0}$ iff $\mathbf{A} = \mathbf{B}^\top \mathbf{B}$ for some $\mathbf{B}$. If $\mathbf{A} \succ \mathbf{0}$, $\mathbf{B}^\top$ can be chosen lower triangular with positive diagonal, $\mathbf{A} = \mathbf{L} \mathbf{L}^\top$ (the **Cholesky factorisation**), and this $\mathbf{L}$ is unique.
- (iii) If $\mathbf{A}, \mathbf{B} \succeq \mathbf{0}$ and $\alpha, \beta \geq 0$ then $\alpha \mathbf{A} + \beta \mathbf{B} \succeq \mathbf{0}$; for any $\mathbf{C}$ of compatible size $\mathbf{C}^\top \mathbf{A} \mathbf{C} \succeq \mathbf{0}$; and $\mathbf{A} + \varepsilon \mathbf{I} \succ \mathbf{0}$ for every $\varepsilon > 0$.

Proof sketch. (i) and (iii) are read off the quadratic form $\sum_i \lambda_i (\mathbf{q}_i^\top \mathbf{x})^2$ and the definition, using $\mathbf{x}^\top \mathbf{C}^\top \mathbf{A} \mathbf{C} \mathbf{x} = (\mathbf{C} \mathbf{x})^\top \mathbf{A} (\mathbf{C} \mathbf{x})$; the eigenvalues of $\mathbf{A}^{-1}$ are $1/\lambda_i$. For (ii), $\mathbf{x}^\top \mathbf{B}^\top \mathbf{B} \mathbf{x} = \|\mathbf{B} \mathbf{x}\|^2 \geq 0$; conversely $\mathbf{B} = \mathbf{\Lambda}^{1/2} \mathbf{Q}^\top$ works, and Gaussian elimination without pivoting produces the unique triangular factor [2, Appendix C]. $\square$

Geometrically, the level set $\mathbf{x}^\top \mathbf{A} \mathbf{x} = 1$ of a PD quadratic form is an ellipsoid with axes $\mathbf{q}_i$ and semi-axes $1/\sqrt{\lambda_i}$; the covariance ellipse of a Gaussian in [Appendix A.3](#) is the level set of the form built from $\mathbf{\Sigma}^{-1}$, so its semi-axes are $\sqrt{\lambda_i(\mathbf{\Sigma})}$.

Example A.7 (A covariance matrix taken apart). The matrix

$$\mathbf{\Sigma} = \begin{bmatrix} 2.0 & 1.2 \\ 1.2 & 1.0 \end{bmatrix} \quad (\text{A.3})$$

is symmetric with $\text{tr} \mathbf{\Sigma} = 3$ and $\det \mathbf{\Sigma} = 0.56$. Its eigenvalues solve $\lambda^2 - 3\lambda + 0.56 = 0$ and are $\lambda_1 = 2.8$ and $\lambda_2 = 0.2$: both positive, so $\mathbf{\Sigma} \succ \mathbf{0}$. The eigenvector of λ_1 is $\mathbf{q}_1 = (3, 2)^\top / \sqrt{13} = (0.832, 0.555)^\top$, at an angle of 33.7° to the x -axis, and $\mathbf{q}_2 = (-2, 3)^\top / \sqrt{13}$ is orthogonal to it. The ellipse $\mathbf{x}^\top \mathbf{\Sigma}^{-1} \mathbf{x} = 1$ has semi-axes $\sqrt{2.8} = 1.673$ along $\mathbf{q}_1$ and $\sqrt{0.2} = 0.447$ along $\mathbf{q}_2$; it is drawn in [Figure A.1](#). The Cholesky factor is $\mathbf{L} = \begin{bmatrix} 1.4142 & 0 \\ 0.8485 & 0.5292 \end{bmatrix}$,

and $\Sigma^{-1} = \frac{1}{0.56} \begin{bmatrix} 1.0 & -1.2 \\ -1.2 & 2.0 \end{bmatrix} = \begin{bmatrix} 1.786 & -2.143 \\ -2.143 & 3.571 \end{bmatrix}$. [Appendix A.3](#) uses this Σ as the covariance of a Gaussian.

A.1.4 Block matrices and the Schur complement

The large matrices of this book come in blocks: the covariance of $(\mathbf{p}, \mathbf{v})$, the joint covariance of a state and a measurement, the KKT matrix of a quadratic program. Blocks multiply like scalars as long as their order is kept,

$$\begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{C} & \mathbf{D} \end{bmatrix} \begin{bmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{bmatrix} = \begin{bmatrix} \mathbf{AE} + \mathbf{BG} & \mathbf{AF} + \mathbf{BH} \\ \mathbf{CE} + \mathbf{DG} & \mathbf{CF} + \mathbf{DH} \end{bmatrix}, \quad (\text{A.4})$$

a block-diagonal matrix is inverted block by block, and the transpose of a block matrix transposes both the arrangement and the blocks.

Definition A.8 (Schur complement). Let $\mathbf{M} = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{C} & \mathbf{D} \end{bmatrix}$ with $\mathbf{A}$ and $\mathbf{D}$ square. If $\mathbf{D}$ is invertible, the **Schur complement** of $\mathbf{D}$ in $\mathbf{M}$ is $\mathbf{S} = \mathbf{A} - \mathbf{BD}^{-1}\mathbf{C}$; if $\mathbf{A}$ is invertible, the Schur complement of $\mathbf{A}$ in $\mathbf{M}$ is $\mathbf{S}' = \mathbf{D} - \mathbf{CA}^{-1}\mathbf{B}$.

Theorem A.9 (Block factorisation, determinant and inverse). If $\mathbf{D}$ is invertible and $\mathbf{S} = \mathbf{A} - \mathbf{BD}^{-1}\mathbf{C}$, then

$$\mathbf{M} = \begin{bmatrix} \mathbf{I} & \mathbf{BD}^{-1} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{S} & \mathbf{0} \\ \mathbf{0} & \mathbf{D} \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{D}^{-1}\mathbf{C} & \mathbf{I} \end{bmatrix}, \quad \det \mathbf{M} = \det \mathbf{S} \det \mathbf{D}, \quad (\text{A.5})$$

and $\mathbf{M}$ is invertible iff $\mathbf{S}$ is, with

$$\mathbf{M}^{-1} = \begin{bmatrix} \mathbf{S}^{-1} & -\mathbf{S}^{-1}\mathbf{BD}^{-1} \\ -\mathbf{D}^{-1}\mathbf{CS}^{-1} & \mathbf{D}^{-1} + \mathbf{D}^{-1}\mathbf{CS}^{-1}\mathbf{BD}^{-1} \end{bmatrix}. \quad (\text{A.6})$$

If instead $\mathbf{A}$ is invertible and $\mathbf{S}' = \mathbf{D} - \mathbf{CA}^{-1}\mathbf{B}$, the same statements hold with the roles of the blocks exchanged; in particular $\det \mathbf{M} = \det \mathbf{A} \det \mathbf{S}'$, $(\mathbf{M}^{-1})_{22} = \mathbf{S}'^{-1}$ and $(\mathbf{M}^{-1})_{11} = \mathbf{A}^{-1} + \mathbf{A}^{-1}\mathbf{BS}'^{-1}\mathbf{CA}^{-1}$.

Proof. Multiplying out the right-hand side of [Equation \(A.5\)](#) with [Equation \(A.4\)](#) gives back $\mathbf{M}$. The outer factors are block-triangular with identity diagonal blocks, so they have determinant 1 and are inverted by negating the off-diagonal block; the middle factor has determinant $\det \mathbf{S} \det \mathbf{D}$ and is inverted block by block. Inverting the three factors in reverse order and multiplying out gives [Equation \(A.6\)](#). The second statement is the first applied to $\mathbf{M}$ with its block rows and block columns swapped. $\square$

In [Example A.7](#) the Schur complement of the $(2, 2)$ entry is $2.0 - 1.2^2/1.0 = 0.56$, which is both $\det \Sigma / \Sigma_{22}$ and the reciprocal of the top-left entry 1.786 of Σ^{-1} , as the theorem says. Reading the theorem in both directions gives the identity with which ?? passes between the covariance and the information form of the Kalman update, and which turns the inversion of a large matrix into that of a small one whenever the update has low rank.

Proposition A.10 (Matrix inversion lemma, Woodbury identity). For $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\mathbf{U} \in \mathbb{R}^{n \times k}$, $\mathbf{C} \in \mathbb{R}^{k \times k}$ and $\mathbf{V} \in \mathbb{R}^{k \times n}$, whenever all inverses below exist,

$$(\mathbf{A} + \mathbf{UCV})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}(\mathbf{C}^{-1} + \mathbf{VA}^{-1}\mathbf{U})^{-1}\mathbf{VA}^{-1}. \quad (\text{A.7})$$

Proof. Apply [Theorem A.9](#) to $\mathbf{M} = \begin{bmatrix} \mathbf{A} & \mathbf{U} \\ \mathbf{V} & \mathbf{C}^{-1} \end{bmatrix}$: eliminating the $(2, 2)$ block gives $(\mathbf{M}^{-1})_{11} = (\mathbf{A} + \mathbf{UCV})^{-1}$, eliminating the $(1, 1)$ block gives $(\mathbf{M}^{-1})_{11} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}(\mathbf{C}^{-1} + \mathbf{VA}^{-1}\mathbf{U})^{-1}\mathbf{VA}^{-1}$, and the two expressions are equal. $\square$

A.1.5 Least squares and the normal equations

Theorem A.11 (Normal equations). Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$. A vector $\mathbf{x}^*$ minimises $\|\mathbf{Ax} - \mathbf{b}\|^2$ iff it satisfies the *normal equations*

$$\mathbf{A}^\top \mathbf{A} \mathbf{x}^* = \mathbf{A}^\top \mathbf{b}. \quad (\text{A.8})$$

If $\mathbf{A}$ has full column rank n , then $\mathbf{A}^\top \mathbf{A} \succ \mathbf{0}$ and the minimiser is unique, $\mathbf{x}^* = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$. For a weight matrix $\mathbf{W} \succ \mathbf{0}$ the minimiser of $(\mathbf{Ax} - \mathbf{b})^\top \mathbf{W} (\mathbf{Ax} - \mathbf{b})$ solves $\mathbf{A}^\top \mathbf{W} \mathbf{A} \mathbf{x}^* = \mathbf{A}^\top \mathbf{W} \mathbf{b}$.

Proof. With $\mathbf{r} = \mathbf{Ax}^* - \mathbf{b}$ and $\mathbf{d} = \mathbf{x} - \mathbf{x}^*$, $\|\mathbf{Ax} - \mathbf{b}\|^2 = \|\mathbf{r}\|^2 + 2\mathbf{d}^\top \mathbf{A}^\top \mathbf{r} + \|\mathbf{Ad}\|^2$. If Equation (A.8) holds, the middle term vanishes for every $\mathbf{d}$ and $\mathbf{x}^*$ is a minimiser; if not, $\mathbf{d} = -t\mathbf{A}^\top \mathbf{r}$ with a small $t > 0$ makes the last two terms sum to a negative number. Full column rank gives $\|\mathbf{Ad}\|^2 > 0$ for $\mathbf{d} \neq \mathbf{0}$, hence uniqueness; the weighted case is the unweighted one for $\mathbf{W}^{1/2}\mathbf{A}$ and $\mathbf{W}^{1/2}\mathbf{b}$, with $\mathbf{W}^{1/2} = \mathbf{Q}\mathbf{\Lambda}^{1/2}\mathbf{Q}^\top$ from Theorem A.4. $\square$

Example A.12 (Fitting a constant-velocity line). A drone is seen at the positions 0.0, 1.1 and 1.9 m at the times $t = 0, 1, 2$ s along one axis. The model $p(t) = p_0 + vt$ is linear in $\mathbf{x} = (p_0, v)^\top$, with rows $(1, t)$ in $\mathbf{A}$ and $\mathbf{b} = (0.0, 1.1, 1.9)^\top$, so $\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}$, $\mathbf{A}^\top \mathbf{b} = (3.0, 4.9)^\top$, and Equation (A.8) gives $p_0 = 0.05$ m and $v = 0.95$ m/s. The residuals $\mathbf{Ax}^* - \mathbf{b} = (0.05, -0.10, 0.05)^\top$ have squared norm 0.015 and are orthogonal to both columns of $\mathbf{A}$, which is what the normal equations say.

Least squares is the quiet workhorse of Parts VI and VII: the Kalman update of ?? is a weighted least-squares compromise between a prediction and a measurement, weighted by the inverses of their covariances, and an MPC problem without inequality constraints (??) is a least-squares problem in the stacked inputs, which the constraints turn into the quadratic program of Appendix A.4.4.

Do not form the inverse

The formulas above are written with $\mathbf{A}^{-1}$ because that is how they are proved, not how they should be computed. Solving $\mathbf{Ax} = \mathbf{b}$ with `numpy.linalg.solve`, or with a Cholesky factor when $\mathbf{A} \succ \mathbf{0}$, is faster and more accurate than forming `numpy.linalg.inv(A)` and multiplying. Two more habits pay off in ????: re-symmetrise a covariance as $\frac{1}{2}(\mathbf{P} + \mathbf{P}^\top)$ after each update, and check its smallest eigenvalue before a Cholesky factorisation, because rounding can turn a PSD matrix into one with a slightly negative eigenvalue.

A.2 Calculus for motion

A.2.1 Gradient, Jacobian, Hessian and the chain rule

Definition A.13 (Derivatives of vector functions). Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ and $\mathbf{g}: \mathbb{R}^n \rightarrow \mathbb{R}^m$ be differentiable. The **gradient** $\nabla f(\mathbf{x}) \in \mathbb{R}^n$ has the entries $\partial f / \partial x_i$; the **Jacobian** $\mathbf{J}_g(\mathbf{x}) \in \mathbb{R}^{m \times n}$ has the entries $\partial g_i / \partial x_j$, one row per output and one column per input, so that the gradient is the transpose of the Jacobian of a scalar function; the **Hessian** $\nabla^2 f(\mathbf{x}) \in \mathbb{R}^{n \times n}$ has the entries $\partial^2 f / \partial x_i \partial x_j$ and is symmetric when f is twice continuously differentiable.

The gradient points in the direction of steepest ascent and is orthogonal to the level set of f through $\mathbf{x}$; the derivative of f along a unit vector $\mathbf{u}$ is $\nabla f(\mathbf{x})^\top \mathbf{u}$. The **chain rule** for a composition is a matrix product: if $\mathbf{h}(\mathbf{x}) = \mathbf{g}(\mathbf{f}(\mathbf{x}))$ then

$$\mathbf{J}_h(\mathbf{x}) = \mathbf{J}_g(\mathbf{f}(\mathbf{x})) \mathbf{J}_f(\mathbf{x}), \quad (\text{A.9})$$

which is how the gradients of a neural network in ?? are propagated backwards through its layers. The derivatives that recur in this book are

$$\begin{aligned}\nabla(\mathbf{b}^\top \mathbf{x}) &= \mathbf{b}, & \nabla\left(\frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x}\right) &= \mathbf{A}\mathbf{x} \quad (\mathbf{A} \text{ symmetric}), \\ \nabla\left(\frac{1}{2}\|\mathbf{x} - \mathbf{c}\|^2\right) &= \mathbf{x} - \mathbf{c}, & \nabla\|\mathbf{x} - \mathbf{c}\| &= \frac{\mathbf{x} - \mathbf{c}}{\|\mathbf{x} - \mathbf{c}\|},\end{aligned}\tag{A.10}$$

together with $\mathbf{J}_{\mathbf{A}\mathbf{x}} = \mathbf{A}$ and $\nabla^2(\frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x}) = \mathbf{A}$. The attractive potential of ?? is the third expression with $\mathbf{c}$ the goal, so its force $-\nabla U$ points at the goal; the repulsive potential is a function of the clearance to an obstacle, and by the chain rule its gradient is a multiple of the unit vector in the fourth expression, which points away from the obstacle.

Proposition A.14 (Taylor expansion). *If $\mathbf{g}: \mathbb{R}^n \rightarrow \mathbb{R}^m$ is twice continuously differentiable near $\mathbf{x}$, then for small δ*

$$\mathbf{g}(\mathbf{x} + \delta) = \mathbf{g}(\mathbf{x}) + \mathbf{J}_{\mathbf{g}}(\mathbf{x})\delta + \mathbf{r}(\delta), \quad \|\mathbf{r}(\delta)\| \leq c \|\delta\|^2 \tag{A.11}$$

for a constant c that depends on the second derivatives of $\mathbf{g}$ near $\mathbf{x}$. For a scalar f the next term is explicit: $f(\mathbf{x} + \delta) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \delta + \frac{1}{2}\delta^\top \nabla^2 f(\mathbf{x})\delta + \mathcal{O}(\|\delta\|^3)$.

Dropping $\mathbf{r}$ in Equation (A.11) is **linearisation**. The extended Kalman filter of ?? replaces its motion and measurement models by their Jacobians $\mathbf{F}$ and $\mathbf{H}$ at the current estimate, and the size of $\mathbf{r}$ over the spread of the estimate decides whether that is good enough or whether the unscented or particle filters of the same chapter are needed; the second-order expansion underlies Newton's method and the convexity test of Appendix A.4.

Example A.15 (Range and bearing of a target). A radar at the origin measures the range r and the bearing φ of a drone with state $\mathbf{x} = (p_x, p_y, v_x, v_y)^\top$:

$$\mathbf{h}(\mathbf{x}) = \begin{bmatrix} \sqrt{p_x^2 + p_y^2} \\ \text{atan2}(p_y, p_x) \end{bmatrix}, \quad \mathbf{H} = \mathbf{J}_{\mathbf{h}}(\mathbf{x}) = \begin{bmatrix} p_x/r & p_y/r & 0 & 0 \\ -p_y/r^2 & p_x/r^2 & 0 & 0 \end{bmatrix}.$$

At $p_x = 3$, $p_y = 4$ (so $r = 5$) the Jacobian is $\mathbf{H} = \begin{bmatrix} 0.6 & 0.8 & 0 & 0 \\ -0.16 & 0.12 & 0 & 0 \end{bmatrix}$ and $\mathbf{h} = (5, 0.9273)^\top$. For the displacement $\delta = (0.1, -0.2, 0, 0)^\top$ the linear prediction $\mathbf{h} + \mathbf{H}\delta = (4.9000, 0.8873)^\top$ differs from the exact value $(4.9041, 0.8865)^\top$ by $(0.0041, -0.0008)^\top$: an error of second order in $\|\delta\| = 0.22$, as Equation (A.11) promises. The zero columns say that the measurement carries no direct information about the velocity; a filter learns the velocity only through the motion model.

Check every Jacobian numerically

Hand-derived Jacobians are the most common source of silent errors in an extended Kalman filter. Compare each column against the central difference $(\mathbf{h}(\mathbf{x} + \varepsilon \mathbf{e}_j) - \mathbf{h}(\mathbf{x} - \varepsilon \mathbf{e}_j))/2\varepsilon$ with $\varepsilon \approx 10^{-6}$ before you trust it; the script of this appendix does exactly that for Example A.15. Two details bite in practice: `atan2` takes the y argument first, and a bearing difference must be wrapped into $(-\pi, \pi]$ before it is used as an innovation, because a linearisation across the $\pm\pi$ cut is meaningless.

A.2.2 The double-integrator step

The drone model of ?? treats the acceleration $\mathbf{a}$ as the input and integrates it twice. If $\mathbf{a}$ is held constant during a step of length Δt , the velocity grows linearly and the position quadratically,

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \mathbf{a} \Delta t, \quad \mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t) \Delta t + \frac{1}{2}\mathbf{a} \Delta t^2, \tag{A.12}$$

because $\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \int_0^{\Delta t} (\mathbf{v}(t) + \mathbf{a}s) ds$. This is exact, not an approximation, for a piecewise-constant input, which is what a digital controller produces. With the state $\mathbf{x} = (\mathbf{p}, \mathbf{v}) \in \mathbb{R}^{2d}$ and the input $\mathbf{u} = \mathbf{a} \in \mathbb{R}^d$ the step is the linear system $\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k$ with

$$\mathbf{A} = \begin{bmatrix} \mathbf{I} & \Delta t \mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} \frac{1}{2} \Delta t^2 \mathbf{I} \\ \Delta t \mathbf{I} \end{bmatrix}, \quad (\text{A.13})$$

where $\mathbf{I}$ is the $d \times d$ identity. ?? takes $\mathbf{A}$ as the transition matrix $\mathbf{F}$ of a constant-velocity target and turns the unknown acceleration into process noise through $\mathbf{B}$, which is where its white-noise-acceleration covariance $\mathbf{Q}$ comes from; ?? stacks N copies of Equation (A.13) into the prediction matrices of its quadratic program; ?? bounds the velocities reachable in one step by $\|\mathbf{v}_{k+1} - \mathbf{v}_k\| \leq a_{\max} \Delta t$, its dynamic window. The explicit Euler step $\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k \Delta t$ drops the term $\frac{1}{2} \mathbf{a} \Delta t^2$ (with $\Delta t = 0.1$ s, $v = 2$ m/s and $a = 1$ m/s²: 0.200 m instead of the exact 0.205 m, at 2.1 m/s afterwards), an error that accumulates over a trajectory.

A.3 Probability and Gaussians

A.3.1 Random vectors, expectation and covariance

A **random vector** $\mathbf{x} \in \mathbb{R}^n$ is a vector whose entries are random variables; it is described by a density $p(\mathbf{x})$ with $p \geq 0$ and $\int p(\mathbf{x}) d\mathbf{x} = 1$, and the probability that $\mathbf{x}$ falls in a region $\mathcal{R}$ is $\mathbb{P}(\mathbf{x} \in \mathcal{R}) = \int_{\mathcal{R}} p(\mathbf{x}) d\mathbf{x}$.

Definition A.16 (Expectation, covariance, cross-covariance). The **expectation** of $\mathbf{x}$ is $\mu = \mathbb{E}[\mathbf{x}] = \int \mathbf{x} p(\mathbf{x}) d\mathbf{x}$, taken entry by entry, and $\mathbb{E}[\mathbf{g}(\mathbf{x})] = \int \mathbf{g}(\mathbf{x}) p(\mathbf{x}) d\mathbf{x}$ for a function $\mathbf{g}$. The **covariance** is

$$\text{Cov}(\mathbf{x}) = \mathbb{E}[(\mathbf{x} - \mu)(\mathbf{x} - \mu)^T] \in \mathbb{R}^{n \times n}, \quad (\text{A.14})$$

whose diagonal entries are the variances $\text{Var}(x_i) = \sigma_i^2$ and whose off-diagonal entries σ_{ij} say how x_i and x_j vary together; $\rho_{ij} = \sigma_{ij}/(\sigma_i \sigma_j) \in [-1, 1]$ is the correlation coefficient. The **cross-covariance** of two random vectors is $\text{Cov}(\mathbf{x}, \mathbf{y}) = \mathbb{E}[(\mathbf{x} - \mu_x)(\mathbf{y} - \mu_y)^T]$, and $\mathbf{x}, \mathbf{y}$ are **independent** if $p(\mathbf{x}, \mathbf{y}) = p(\mathbf{x})p(\mathbf{y})$, in which case $\text{Cov}(\mathbf{x}, \mathbf{y}) = \mathbf{0}$.

Proposition A.17 (Rules for means and covariances). *Expectation is linear, $\mathbb{E}[\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} + \mathbf{c}] = \mathbf{A}\mathbb{E}[\mathbf{x}] + \mathbf{B}\mathbb{E}[\mathbf{y}] + \mathbf{c}$, with no assumption about dependence. Every covariance matrix is symmetric and PSD. For fixed $\mathbf{A}$ and $\mathbf{b}$,*

$$\text{Cov}(\mathbf{A}\mathbf{x} + \mathbf{b}) = \mathbf{A} \text{Cov}(\mathbf{x}) \mathbf{A}^T, \quad (\text{A.15})$$

and if $\mathbf{x}$ and $\mathbf{y}$ are independent, $\text{Cov}(\mathbf{x} + \mathbf{y}) = \text{Cov}(\mathbf{x}) + \text{Cov}(\mathbf{y})$.

Proof. Linearity is linearity of the integral. For any fixed $\mathbf{a}$, $\mathbf{a}^T \text{Cov}(\mathbf{x}) \mathbf{a} = \mathbb{E}[(\mathbf{a}^T(\mathbf{x} - \mu))^2] = \text{Var}(\mathbf{a}^T \mathbf{x}) \geq 0$, which is the definition of PSD; symmetry is read off Equation (A.14). Since $\mathbf{A}\mathbf{x} + \mathbf{b} - \mathbb{E}[\mathbf{A}\mathbf{x} + \mathbf{b}] = \mathbf{A}(\mathbf{x} - \mu)$, Equation (A.15) follows by pulling $\mathbf{A}$ and $\mathbf{A}^T$ out of the expectation. For the sum, expand $(\mathbf{x} + \mathbf{y} - \mu_x - \mu_y)(\mathbf{x} + \mathbf{y} - \mu_x - \mu_y)^T$; the two cross terms are the cross-covariances, which vanish under independence. $\square$

Equation (A.15) is the single most used formula of ????: pushing a covariance through the linear model $\mathbf{F}$ gives $\mathbf{F}\mathbf{P}\mathbf{F}^T$, and adding independent process noise adds $\mathbf{Q}$. Note that it needs no Gaussian assumption at all.

A.3.2 The multivariate Gaussian

Definition A.18 (Multivariate Gaussian). A random vector $\mathbf{x} \in \mathbb{R}^n$ is **Gaussian** with mean μ and covariance $\Sigma \succ \mathbf{0}$, written $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$, if its density is

$$\mathcal{N}(\mathbf{x}; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} \sqrt{\det \Sigma}} \exp\left(-\frac{1}{2} d^2(\mathbf{x})\right), \quad d^2(\mathbf{x}) = (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu), \quad (\text{A.16})$$

where $d(\mathbf{x})$ is the **Mahalanobis distance** of $\mathbf{x}$ from μ . Then $\mathbb{E}[\mathbf{x}] = \mu$ and $\text{Cov}(\mathbf{x}) = \Sigma$.

The density is constant on the ellipsoids $d^2(\mathbf{x}) = k^2$. By [Theorem A.4](#) applied to $\Sigma = \mathbf{Q}\mathbf{A}\mathbf{Q}^\top$, the ellipsoid $\mathcal{E}_k$ of ?? is centred at μ , its axes are the eigenvectors $\mathbf{q}_i$ and its semi-axes are $k\sqrt{\lambda_i}$: a large eigenvalue is a direction of large uncertainty. In two dimensions the probability mass inside $\mathcal{E}_k$ is $1 - e^{-k^2/2}$, that is 39.3 % for $k = 1$ and 86.5 % for $k = 2$, a consequence of [Theorem A.23](#) below. A singular $\Sigma \succeq \mathbf{0}$ has no density, but the two theorems that follow still hold for it.

Theorem A.19 (Affine transformations of Gaussians). If $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$ in $\mathbb{R}^n$ and $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b}$ with $\mathbf{A} \in \mathbb{R}^{m \times n}$, then $\mathbf{y} \sim \mathcal{N}(\mathbf{A}\mu + \mathbf{b}, \mathbf{A}\Sigma\mathbf{A}^\top)$.

Proof. The mean and the covariance follow from [Proposition A.17](#). For the shape, take first $m = n$ and $\mathbf{A}$ invertible: substituting $\mathbf{x} = \mathbf{A}^{-1}(\mathbf{y} - \mathbf{b})$ into [Equation \(A.16\)](#) and multiplying by the change-of-variables factor $|\det \mathbf{A}^{-1}|$ gives exactly the density $\mathcal{N}(\mathbf{y}; \mathbf{A}\mu + \mathbf{b}, \mathbf{A}\Sigma\mathbf{A}^\top)$, because $(\mathbf{A}\Sigma\mathbf{A}^\top)^{-1} = \mathbf{A}^{-\top}\Sigma^{-1}\mathbf{A}^{-1}$ and $\det(\mathbf{A}\Sigma\mathbf{A}^\top) = (\det \mathbf{A})^2 \det \Sigma$. A non-square $\mathbf{A}$ of full row rank is extended to an invertible matrix, and the extra coordinates are integrated out with the marginalisation half of [Theorem A.20](#), whose proof does not use this theorem; the remaining cases follow with characteristic functions [8, Chapter 4]. $\square$

In particular the sum of independent Gaussians is Gaussian with the sums of the means and of the covariances (apply $\mathbf{A} = [\mathbf{I} \ \mathbf{I}]$ to the joint vector, whose covariance is block-diagonal), which is how process noise enters the prediction step of ??.

Theorem A.20 (Marginal and conditional of a joint Gaussian). Let $\mathbf{x} = (\mathbf{x}_a, \mathbf{x}_b)$ be jointly Gaussian,

$$\begin{bmatrix} \mathbf{x}_a \\ \mathbf{x}_b \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu_a \\ \mu_b \end{bmatrix}, \begin{bmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{bmatrix}\right), \quad \Sigma_{ba} = \Sigma_{ab}^\top. \quad (\text{A.17})$$

Then the **marginal** of $\mathbf{x}_b$ is $\mathcal{N}(\mu_b, \Sigma_{bb})$ (and likewise for $\mathbf{x}_a$), and the **conditional** density of $\mathbf{x}_a$ given $\mathbf{x}_b$ is Gaussian,

$$\mathbf{x}_a | \mathbf{x}_b \sim \mathcal{N}\left(\mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(\mathbf{x}_b - \mu_b), \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}\right). \quad (\text{A.18})$$

The conditional mean is affine in the observed value; the conditional covariance is the Schur complement of Σ_{bb} , does not depend on the observed value, and is never larger than Σ_{aa} .

Proof. Write $\mathbf{d}_a = \mathbf{x}_a - \mu_a$, $\mathbf{d}_b = \mathbf{x}_b - \mu_b$, $\mathbf{S} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$ and $\mathbf{\Lambda} = \Sigma^{-1}$ with blocks $\Lambda_{aa}, \Lambda_{ab}, \Lambda_{ba}, \Lambda_{bb}$. By [Theorem A.9](#) with $\mathbf{D} = \Sigma_{bb}$,

$$\Lambda_{aa} = \mathbf{S}^{-1}, \quad \Lambda_{ab} = -\mathbf{S}^{-1}\Sigma_{ab}\Sigma_{bb}^{-1}, \quad \det \Sigma = \det \mathbf{S} \det \Sigma_{bb}.$$

The exponent of the joint density is $-\frac{1}{2}(\mathbf{d}_a^\top \Lambda_{aa} \mathbf{d}_a + 2\mathbf{d}_a^\top \Lambda_{ab} \mathbf{d}_b + \mathbf{d}_b^\top \Lambda_{bb} \mathbf{d}_b)$. Completing the square in $\mathbf{d}_a$ with $\mathbf{m} = -\Lambda_{aa}^{-1}\Lambda_{ab}\mathbf{d}_b = \Sigma_{ab}\Sigma_{bb}^{-1}\mathbf{d}_b$ turns it into

$$-\frac{1}{2}(\mathbf{d}_a - \mathbf{m})^\top \mathbf{S}^{-1}(\mathbf{d}_a - \mathbf{m}) - \frac{1}{2}\mathbf{d}_b^\top \left(\Lambda_{bb} - \Lambda_{ba}\Lambda_{aa}^{-1}\Lambda_{ab}\right) \mathbf{d}_b,$$

and the matrix in the second term is the Schur complement of Λ_{aa} in $\mathbf{\Lambda}$, which by [Theorem A.9](#) applied to $\mathbf{\Lambda}$ (whose inverse is Σ) equals Σ_{bb}^{-1} . Splitting the normalising constant with

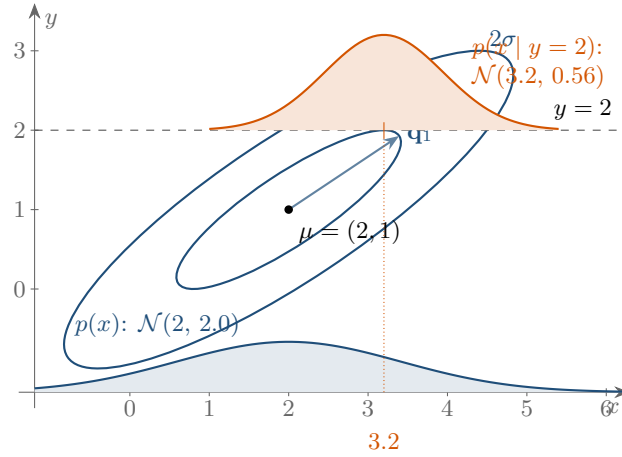


Figure A.1. Conditioning the Gaussian of Example A.22. The 1σ and 2σ ellipses of the joint density are tilted along the eigenvector $\mathbf{q}_1$ of Σ . Observing $y = 2$ restricts the density to the dashed slice; along it, the conditional density of x (orange) is centred at 3.2, to the right of the prior mean 2, and is narrower than the marginal density of x (blue, drawn along the bottom) because knowing y removes the part of the uncertainty of x that is correlated with y .

$\det \Sigma = \det \mathbf{S} \det \Sigma_{bb}$ and $(2\pi)^n = (2\pi)^{n_a} (2\pi)^{n_b}$ gives the exact factorisation

$$p(\mathbf{x}_a, \mathbf{x}_b) = \mathcal{N}(\mathbf{x}_a; \mu_a + \mathbf{m}, \mathbf{S}) \cdot \mathcal{N}(\mathbf{x}_b; \mu_b, \Sigma_{bb}). \quad (\text{A.19})$$

Integrating over $\mathbf{x}_a$ removes the first factor, which is a density in $\mathbf{x}_a$, and leaves the marginal of $\mathbf{x}_b$. Dividing by that marginal gives $p(\mathbf{x}_a | \mathbf{x}_b) = p(\mathbf{x}_a, \mathbf{x}_b) / p(\mathbf{x}_b)$, the first factor, which is Equation (A.18). Finally $\Sigma_{aa} - \mathbf{S} = \Sigma_{ab} \Sigma_{bb}^{-1} \Sigma_{ba} \succeq \mathbf{0}$ by Proposition A.6(iii). $\square$

Corollary A.21 (Linear-Gaussian measurement update). *Let $\mathbf{x} \sim \mathcal{N}(\mu, \mathbf{P})$ and $\mathbf{z} = \mathbf{H}\mathbf{x} + \mathbf{v}$ with $\mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$ independent of $\mathbf{x}$. Then $(\mathbf{x}, \mathbf{z})$ is jointly Gaussian with $\text{Cov}(\mathbf{z}) = \mathbf{H}\mathbf{P}\mathbf{H}^\top + \mathbf{R} =: \mathbf{S}$ and $\text{Cov}(\mathbf{x}, \mathbf{z}) = \mathbf{P}\mathbf{H}^\top$, and*

$$\mathbf{x} | \mathbf{z} \sim \mathcal{N}(\mu + \mathbf{K}(\mathbf{z} - \mathbf{H}\mu), \mathbf{P} - \mathbf{K}\mathbf{H}\mathbf{P}), \quad \mathbf{K} = \mathbf{P}\mathbf{H}^\top \mathbf{S}^{-1}. \quad (\text{A.20})$$

Proof. $(\mathbf{x}, \mathbf{z})$ is the image of the Gaussian vector $(\mathbf{x}, \mathbf{v})$, whose covariance is block-diagonal, under the linear map $\begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{H} & \mathbf{I} \end{bmatrix}$, hence Gaussian by Theorem A.19; its blocks follow from Proposition A.17. Insert $\Sigma_{ab} = \mathbf{P}\mathbf{H}^\top$ and $\Sigma_{bb} = \mathbf{S}$ into Equation (A.18). $\square$

Equation (A.20) is the update step of the Kalman filter with gain $\mathbf{K}$: ?? derives it from this corollary, and Proposition A.10 turns $\mathbf{P} - \mathbf{K}\mathbf{H}\mathbf{P}$ into the information form $(\mathbf{P}^{-1} + \mathbf{H}^\top \mathbf{R}^{-1} \mathbf{H})^{-1}$, in which the precisions of prior and measurement simply add.

Example A.22 (Conditioning the Gaussian of Example A.7). Let $(x, y) \sim \mathcal{N}(\mu, \Sigma)$ with $\mu = (2, 1)^\top$ and Σ from Equation (A.3), and suppose $y = 2$ is observed. Equation (A.18) with $a = x$ and $b = y$ gives

$$\mathbb{E}[x | y = 2] = 2 + \frac{1.2}{1.0}(2 - 1) = 3.2, \quad \text{Var}(x | y = 2) = 2.0 - \frac{1.2^2}{1.0} = 0.56.$$

Before the observation $x \sim \mathcal{N}(2, 2.0)$, with standard deviation 1.414; afterwards $x | y \sim \mathcal{N}(3.2, 0.56)$, with standard deviation 0.748. The positive correlation $\rho = 1.2 / \sqrt{2.0 \cdot 1.0} = 0.85$ means that a y above its mean shifts the belief about x upwards, and the variance shrinks by the factor $1 - \rho^2 = 0.28$. Figure A.1 shows the ellipses, the slice $y = 2$ and both densities.

Table A.2. Gaussian identities. $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$, blocks as in Equation (A.17); “ $\propto$ ” means equality up to a factor that does not depend on $\mathbf{x}$.

Identity	Remark
$\mathbf{Ax} + \mathbf{b} \sim \mathcal{N}(\mathbf{A}\mu + \mathbf{b}, \mathbf{A}\Sigma\mathbf{A}^\top)$	Theorem A.19 ; prediction step
$\mathbf{x}_a \sim \mathcal{N}(\mu_a, \Sigma_{aa})$	marginal: drop the other blocks
$\mathbf{x}_a \mathbf{x}_b \sim \mathcal{N}(\mu_{a b}, \Sigma_{a b}), \mu_{a b} = \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(\mathbf{x}_b - \mu_b),$ $\Sigma_{a b} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$	Theorem A.20 ; update step
$\mathbf{x} + \mathbf{y} \sim \mathcal{N}(\mu_x + \mu_y, \Sigma_x + \Sigma_y)$	$\mathbf{x}, \mathbf{y}$ independent
$\mathcal{N}(\mathbf{x}; \mathbf{a}, \mathbf{A})\mathcal{N}(\mathbf{x}; \mathbf{b}, \mathbf{B}) \propto \mathcal{N}(\mathbf{x}; \mathbf{c}, \mathbf{C}), \mathbf{C} = (\mathbf{A}^{-1} + \mathbf{B}^{-1})^{-1},$ $\mathbf{c} = \mathbf{C}(\mathbf{A}^{-1}\mathbf{a} + \mathbf{B}^{-1}\mathbf{b})$	precisions add; information form of the update
$d^2(\mathbf{x}) = (\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu) \sim \chi_n^2$	Theorem A.23 ; gating
$\mathbf{x} = \mu + \mathbf{L}\xi, \mathbf{L}\mathbf{L}^\top = \Sigma, \xi \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$	how to draw a sample

A.3.3 Bayes’ rule, total probability and the recursive filter

For two random vectors the **chain rule** $p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z} | \mathbf{x})p(\mathbf{x})$ defines the conditional density, and integrating it over $\mathbf{x}$ gives the law of **total probability**, $p(\mathbf{z}) = \int p(\mathbf{z} | \mathbf{x})p(\mathbf{x}) d\mathbf{x}$. Reading the chain rule in both directions gives **Bayes’ rule**

$$p(\mathbf{x} | \mathbf{z}) = \frac{p(\mathbf{z} | \mathbf{x})p(\mathbf{x})}{p(\mathbf{z})} = \frac{p(\mathbf{z} | \mathbf{x})p(\mathbf{x})}{\int p(\mathbf{z} | \mathbf{x}')p(\mathbf{x}') d\mathbf{x}'}, \quad (\text{A.21})$$

which turns a **prior** $p(\mathbf{x})$ and a **likelihood** $p(\mathbf{z} | \mathbf{x})$ into the **posterior** $p(\mathbf{x} | \mathbf{z})$; the denominator is a constant that makes the posterior integrate to one. For discrete events the integrals are sums. For example, a detector that flags an intruder with probability 0.9 when one is present and 0.2 when none is, on a day with intruders present 10 % of the time, raises the probability of an intruder on a flag only to $0.09/(0.09 + 0.18) = 1/3$: false alarms on the common case outnumber true detections on the rare one.

Every filter of **????** is these two rules applied over and over: with the Markov assumption $p(\mathbf{x}_k | \mathbf{x}_{k-1}, \mathbf{z}_{1:k-1}) = p(\mathbf{x}_k | \mathbf{x}_{k-1})$ and measurements that depend only on the current state, the belief about $\mathbf{x}_k$ given $\mathbf{z}_{1:k}$ is obtained in two steps,

$$p(\mathbf{x}_k | \mathbf{z}_{1:k-1}) = \int p(\mathbf{x}_k | \mathbf{x}_{k-1})p(\mathbf{x}_{k-1} | \mathbf{z}_{1:k-1}) d\mathbf{x}_{k-1} \quad (\text{predict: total probability}), \quad (\text{A.22})$$

$$p(\mathbf{x}_k | \mathbf{z}_{1:k}) \propto p(\mathbf{z}_k | \mathbf{x}_k)p(\mathbf{x}_k | \mathbf{z}_{1:k-1}) \quad (\text{update: Bayes’ rule}). \quad (\text{A.23})$$

When the motion and measurement models are linear with Gaussian noise, both steps map Gaussians to Gaussians ([Theorem A.19](#) and [Corollary A.21](#)) and the recursion is the Kalman filter; when they are not, **??** approximates the two integrals by linearisation, by sigma points or by samples [8, 9].

A.3.4 Monte Carlo estimation

Many quantities in this book are expectations without a closed form, such as the probability that a predicted trajectory hits an obstacle or the collision rate of a planner over random scenarios. The **Monte Carlo** estimate of $\mathbb{E}[g(\mathbf{x})]$ from N independent samples $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}$ drawn from p is

$$\hat{g}_N = \frac{1}{N} \sum_{i=1}^N g(\mathbf{x}^{(i)}), \quad \mathbb{E}[\hat{g}_N] = \mathbb{E}[g(\mathbf{x})], \quad \text{Var}(\hat{g}_N) = \frac{\text{Var}(g(\mathbf{x}))}{N}. \quad (\text{A.24})$$

The estimate is unbiased and its standard error shrinks like $1/\sqrt{N}$ whatever the dimension of $\mathbf{x}$. For a probability $\mathbb{P}(\mathbf{x} \in \mathcal{R})$ take g the indicator of $\mathcal{R}$; then the standard error is $\sqrt{\hat{p}(1-\hat{p})/N}$, the error bar to attach to a collision rate estimated from N random scenarios (??). To draw from $\mathcal{N}(\mu, \Sigma)$, draw $\xi \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and set $\mathbf{x} = \mu + \mathbf{L}\xi$ with the Cholesky factor $\mathbf{L}\mathbf{L}^\top = \Sigma$ (Theorem A.19 in reverse). Drawing $N = 10\,000$ samples this way from the Gaussian of Example A.22 and counting those with $d^2 \leq 4$ gives $\hat{p} = 0.8695$ with standard error 0.0034, within 1.5 standard errors of the exact value $1 - e^{-2} = 0.8647$ of Theorem A.23 below. When samples can only be drawn from another density q , **importance sampling** weights them by $w^{(i)} \propto p(\mathbf{x}^{(i)})/q(\mathbf{x}^{(i)})$, normalised to sum to one; the particle filter of ?? is a sequential version of this idea [4].

A.3.5 Mahalanobis distance and chi-square gating

Theorem A.23 (Mahalanobis distance of a Gaussian vector). *If $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$ in $\mathbb{R}^n$ with $\Sigma \succ \mathbf{0}$, then $d^2(\mathbf{x}) = (\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu)$ has the **chi-square distribution** with n degrees of freedom, χ_n^2 , whose mean is n . For $n = 2$, $\mathbb{P}(d^2 \leq \gamma) = 1 - e^{-\gamma/2}$.*

Proof. Let $\mathbf{L}\mathbf{L}^\top = \Sigma$ and $\xi = \mathbf{L}^{-1}(\mathbf{x} - \mu)$. By Theorem A.19, $\xi \sim \mathcal{N}(\mathbf{0}, \mathbf{L}^{-1}\Sigma\mathbf{L}^{-\top}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, so its entries are n independent standard normal variables, and $d^2 = \xi^\top \xi = \sum_i \xi_i^2$ is by definition χ_n^2 distributed; $\mathbb{E}[\xi_i^2] = 1$ gives the mean. For $n = 2$ the density of (ξ_1, ξ_2) is $\frac{1}{2\pi}e^{-\varrho^2/2}$ in polar coordinates (ϱ, ϕ) , and $\mathbb{P}(d^2 \leq \gamma) = \int_0^{\sqrt{\gamma}} e^{-\varrho^2/2} \varrho d\varrho = 1 - e^{-\gamma/2}$. $\square$

The theorem is the basis of **gating**: a filter that expects a measurement with innovation $\mathbf{y} = \mathbf{z} - \hat{\mathbf{z}}$ of covariance $\mathbf{S}$ accepts it only if $\mathbf{y}^\top \mathbf{S}^{-1} \mathbf{y} \leq \gamma$, with γ the quantile of χ_n^2 at the chosen confidence p : for $n = 2$ the 95 % and 99 % gates are $\gamma = -2 \ln(1 - p) = 5.991$ and 9.210, for $n = 1$ they are 3.841 and 6.635, and for $n = 3$ they are 7.815 and 11.345. A true measurement is then wrongly rejected with probability $1 - p$, and a detection from another target or from clutter is rejected whenever it lands outside the ellipsoid [1]. This is the gate of ????; the same quantiles size the uncertainty-inflated constraints of ??, where an obstacle known only as a Gaussian is enlarged to the ellipse that holds a required probability mass.

A.4 Optimization

A.4.1 Convex sets and convex functions

Definition A.24 (Convex set, convex function). A set $\mathcal{C} \subseteq \mathbb{R}^n$ is **convex** if the segment between any two of its points stays inside it: $\mathbf{x}, \mathbf{y} \in \mathcal{C}$ and $\theta \in [0, 1]$ imply $\theta\mathbf{x} + (1 - \theta)\mathbf{y} \in \mathcal{C}$. A function $f: \mathcal{C} \rightarrow \mathbb{R}$ on a convex set is **convex** if its graph lies below every chord,

$$f(\theta\mathbf{x} + (1 - \theta)\mathbf{y}) \leq \theta f(\mathbf{x}) + (1 - \theta)f(\mathbf{y}) \quad \text{for all } \mathbf{x}, \mathbf{y} \in \mathcal{C}, \theta \in [0, 1], \quad (\text{A.25})$$

strictly convex if the inequality is strict for $\mathbf{x} \neq \mathbf{y}$ and $\theta \in (0, 1)$, and concave if $-f$ is convex.

Half-spaces, balls, ellipsoids and affine subspaces are convex, and any intersection of convex sets is convex: the velocities allowed by all ORCA half-planes in ?? form a convex polygon, whereas the collision-free velocities outside a velocity obstacle of ?? do not form a convex set (Figure A.2). Linear functions, norms, $\|\mathbf{x} - \mathbf{c}\|^2$ and $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ with $\mathbf{A} \succeq \mathbf{0}$ are convex, as are non-negative sums and pointwise maxima of convex functions, and the sublevel sets $\{\mathbf{x} : f(\mathbf{x}) \leq \alpha\}$ of a convex function are convex. What makes convexity the dividing line of optimization is the following fact.

Proposition A.25 (Local minima of convex functions are global). *Let f be convex on a convex set $\mathcal{C}$ and let $\mathbf{x}^*$ be a local minimiser: $f(\mathbf{x}^*) \leq f(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{C}$ within some distance*

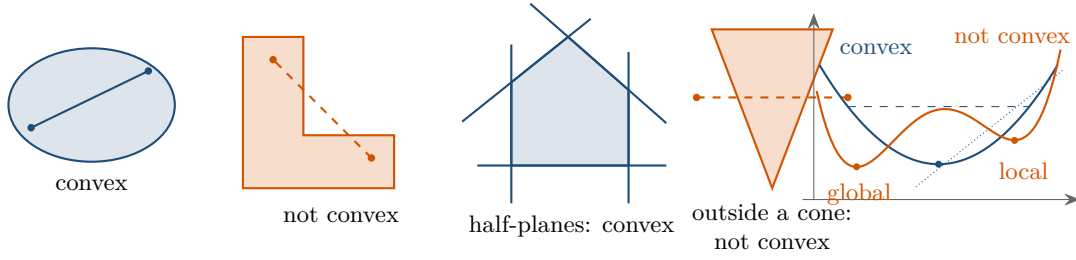


Figure A.2. Convexity. Left: in a convex set every chord stays inside; in the non-convex set the dashed chord leaves it. Middle: an intersection of half-planes, the situation of ORCA in ??, is convex, while the set of velocities outside a cone, the situation of a velocity obstacle in ??, is not. Right: a convex function lies below its chords and above its tangents, so its only stationary point is the global minimum; the non-convex function has a local minimum in which gradient descent can stop.

$\varepsilon > 0$ of $\mathbf{x}^*$. Then $f(\mathbf{x}^*) \leq f(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{C}$. If f is differentiable on $\mathbb{R}^n$, $\mathbf{x}^*$ is a global minimiser iff $\nabla f(\mathbf{x}^*) = \mathbf{0}$.

Proof. Suppose $f(\mathbf{y}) < f(\mathbf{x}^*)$ for some $\mathbf{y} \in \mathcal{C}$. The points $\mathbf{x}_\theta = \theta\mathbf{y} + (1 - \theta)\mathbf{x}^*$ lie in $\mathcal{C}$ and satisfy $f(\mathbf{x}_\theta) \leq \theta f(\mathbf{y}) + (1 - \theta)f(\mathbf{x}^*) < f(\mathbf{x}^*)$ for every $\theta \in (0, 1]$; for θ small enough $\mathbf{x}_\theta$ is within ε of $\mathbf{x}^*$, contradicting local minimality. For the second claim, a differentiable convex function satisfies $f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top(\mathbf{y} - \mathbf{x})$ for all $\mathbf{x}, \mathbf{y}$, that is, its graph lies above its tangent planes [2, Section 3.1], so a zero gradient makes $\mathbf{x}$ a global minimiser; conversely, a minimiser of a differentiable function has zero gradient. $\square$

For twice differentiable f the test is the Hessian: f is convex on $\mathbb{R}^n$ iff $\nabla^2 f(\mathbf{x}) \succeq \mathbf{0}$ everywhere [2, Section 3.1]. This is why the quadratic costs of ?? are built from PSD weights, and why the potential fields of ??, whose repulsive terms are not convex, can trap a drone in a local minimum.

A.4.2 Optimality conditions: Lagrange multipliers and KKT

The general constrained problem is

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad \text{subject to} \quad g_i(\mathbf{x}) \leq 0 \quad (i = 1, \dots, m), \quad h_j(\mathbf{x}) = 0 \quad (j = 1, \dots, p), \quad (\text{A.26})$$

with differentiable f , g_i and h_j . Its **Lagrangian** attaches a multiplier to every constraint, $\mathcal{L}(\mathbf{x}, \lambda, \nu) = f(\mathbf{x}) + \sum_i \lambda_i g_i(\mathbf{x}) + \sum_j \nu_j h_j(\mathbf{x})$.

Theorem A.26 (Karush–Kuhn–Tucker conditions). *If $\mathbf{x}^*$ is a local minimiser of Equation (A.26) and the constraints satisfy a constraint qualification at $\mathbf{x}^*$ (for instance, the gradients of the active constraints are linearly independent, or all constraints are affine), then there exist multipliers $\lambda^* \in \mathbb{R}^m$ and $\nu^* \in \mathbb{R}^p$ with*

$$\nabla f(\mathbf{x}^*) + \sum_i \lambda_i^* \nabla g_i(\mathbf{x}^*) + \sum_j \nu_j^* \nabla h_j(\mathbf{x}^*) = \mathbf{0} \quad (\text{stationarity}), \quad (\text{A.27})$$

$$g_i(\mathbf{x}^*) \leq 0, \quad h_j(\mathbf{x}^*) = 0 \quad (\text{primal feasibility}), \quad (\text{A.28})$$

$$\lambda_i^* \geq 0 \quad (\text{dual feasibility}), \quad (\text{A.29})$$

$$\lambda_i^* g_i(\mathbf{x}^*) = 0 \quad (\text{complementary slackness}). \quad (\text{A.30})$$

If f and the g_i are convex and the h_j affine, these conditions are also sufficient: every point that satisfies them is a global minimiser.

The proof is in Nocedal and Wright [6, Chapter 12] and, for the convex case, in Boyd and Vandenberghe [2, Chapter 5]. At the optimum the negative gradient of the cost is balanced by

the normals of the *active* constraints, those with $g_i(\mathbf{x}^*) = 0$, which by Equation (A.30) are the only ones that can carry a positive multiplier; λ_i^* measures how much the optimal cost would drop if constraint i were relaxed by one unit. This is the theory behind the soft constraints of ?? : a slack variable with a linear penalty reproduces the hard constraint exactly, whenever it is feasible, as soon as the penalty weight exceeds the multiplier [6, Chapter 17]. With equality constraints only, the conditions reduce to the classical Lagrange multipliers.

A.4.3 Linear programs

Definition A.27 (Linear program). A **linear program** (LP) minimises a linear function over a polyhedron:

$$\min_{\mathbf{z} \in \mathbb{R}^n} \mathbf{c}^\top \mathbf{z} \quad \text{subject to} \quad \mathbf{A}\mathbf{z} \leq \mathbf{b}, \quad P = \{\mathbf{z} : \mathbf{A}\mathbf{z} \leq \mathbf{b}\} \text{ the feasible polyhedron.} \quad (\text{A.31})$$

Equality constraints, sign constraints $\mathbf{z} \geq \mathbf{0}$ and maximisation are all reduced to this form by writing $\mathbf{a}^\top \mathbf{z} = b$ as two inequalities, $z_j \geq 0$ as $-z_j \leq 0$ and $\max \mathbf{c}^\top \mathbf{z}$ as $\min(-\mathbf{c})^\top \mathbf{z}$. Simplex solvers and most textbooks use instead the **standard form** $\min \mathbf{c}^\top \mathbf{z}$ subject to $\mathbf{A}\mathbf{z} = \mathbf{b}$, $\mathbf{z} \geq \mathbf{0}$, reached from Equation (A.31) by adding a *slack* variable $s_i \geq 0$ to every inequality, $\mathbf{a}_i^\top \mathbf{z} + s_i = b_i$, and by splitting a free variable into $z_j = z_j^+ - z_j^-$ with $z_j^\pm \geq 0$; `scipy.optimize.linprog` accepts inequalities, equalities and bounds directly.

An LP is convex, so Theorem A.26 applies with $\nabla g_i = \mathbf{a}_i$: at an optimum, $-\mathbf{c}$ is a non-negative combination of the normals of the active constraints. In two dimensions this is a picture (Figure A.3): P is a polygon, the level lines $\mathbf{c}^\top \mathbf{z} = \text{const}$ are parallel, and sliding a level line in the direction $-\mathbf{c}$ as far as P allows ends at a vertex, or along a whole edge when the level lines are parallel to it.

Theorem A.28 (Vertex optimality). *If the LP Equation (A.31) has an optimal solution and P has at least one vertex, then some vertex of P is optimal. Otherwise the LP is infeasible ($P = \emptyset$) or unbounded ($\mathbf{c}^\top \mathbf{z} \rightarrow -\infty$ on P).*

A proof is in Nocedal and Wright [6, Chapter 13]. The simplex method walks from vertex to adjacent vertex, improving the objective at every step until no neighbour is better, which by convexity is a global optimum; interior-point methods approach the optimum through the inside of P and are polynomial in the worst case. ?? calls such solvers through `scipy.optimize`, and branch and bound (Appendix A.4.5) solves a sequence of LPs. The velocity choice of ORCA in ?? is the same geometry: the velocity closest to the preferred one inside an intersection of half-planes, found by a small linear-programming routine that adds one half-plane at a time.

Example A.29 (A two-variable LP). Maximise $2z_1 + 3z_2$ subject to $z_1 + z_2 \leq 4$, $z_1 + 3z_2 \leq 6$ and $z_1, z_2 \geq 0$. The polygon P has the vertices $(0, 0)$, $(4, 0)$, $(3, 1)$ and $(0, 2)$, where the objective takes the values 0, 8, 9 and 6; the optimum is $\mathbf{z}^* = (3, 1)$ with value 9 (Figure A.3). At $\mathbf{z}^*$ both structural constraints are active, and the stationarity condition for the maximisation reads $\mathbf{c} = \lambda_1(1, 1)^\top + \lambda_2(1, 3)^\top$, solved by $\lambda_1 = 1.5$ and $\lambda_2 = 0.5$: both non-negative, as required, so $\mathbf{c}$ lies in the cone spanned by the two active normals and no feasible direction improves the objective. Relaxing the first constraint to $z_1 + z_2 \leq 5$ raises the optimum by exactly $\lambda_1 = 1.5$, to 10.5 at $(4.5, 0.5)$.

A.4.4 Quadratic programs

A **quadratic program** (QP) has a quadratic objective and linear constraints,

$$\min_{\mathbf{z}} \frac{1}{2} \mathbf{z}^\top \mathbf{H} \mathbf{z} + \mathbf{f}^\top \mathbf{z} \quad \text{subject to} \quad \mathbf{l} \leq \mathbf{G} \mathbf{z} \leq \mathbf{u}, \quad (\text{A.32})$$

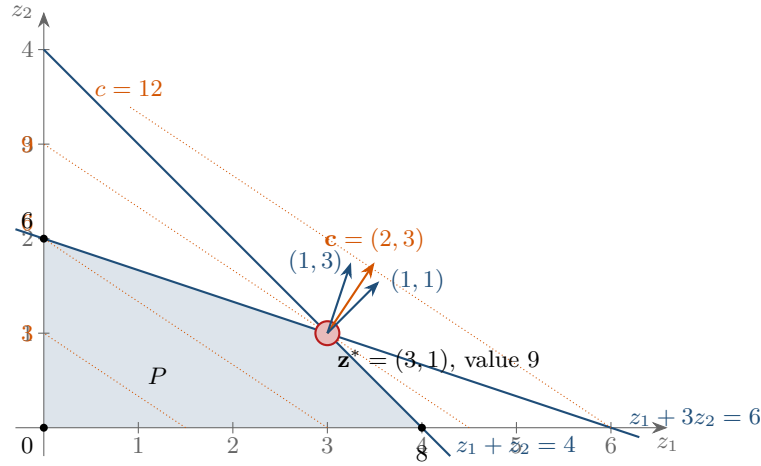


Figure A.3. The LP of Example A.29. The shaded polygon is the feasible region P , cut out by four half-planes; the dotted lines are level lines of the objective $2z_1 + 3z_2$. Sliding a level line in the direction of $\mathbf{c}$ until it is about to leave P stops at the vertex $(3, 1)$, where $\mathbf{c}$ lies between the normals of the two active constraints.

which is the form of the MPC problem of ???. It is convex iff $\mathbf{H} \succeq \mathbf{0}$, and then Theorem A.26 is necessary and sufficient. Without constraints the solution is the linear system $\mathbf{H}\mathbf{z} = -\mathbf{f}$, unique when $\mathbf{H} \succ \mathbf{0}$; the least-squares problem of Theorem A.11 is the case $\mathbf{H} = 2\mathbf{A}^\top \mathbf{A}$, $\mathbf{f} = -2\mathbf{A}^\top \mathbf{b}$. With equality constraints $\mathbf{G}\mathbf{z} = \mathbf{g}$ only, the KKT conditions are the single linear system

$$\begin{bmatrix} \mathbf{H} & \mathbf{G}^\top \\ \mathbf{G} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{z} \\ \nu \end{bmatrix} = \begin{bmatrix} -\mathbf{f} \\ \mathbf{g} \end{bmatrix}, \quad (\text{A.33})$$

a block matrix whose Schur complement $-\mathbf{G}\mathbf{H}^{-1}\mathbf{G}^\top$ (Theorem A.9) is what a solver eliminates first. Inequality constraints are handled by active-set methods, which guess the active set and solve Equation (A.33) for it, by interior-point methods [6], or by operator-splitting methods such as ADMM, which alternate projections onto the constraints with linear solves.

A.4.5 Mixed-integer programs, branch and bound, big-M

A **mixed-integer linear program** (MILP) is an LP in which some variables are restricted to integers, most often to the binary values $b \in \{0, 1\}$. The integrality constraint destroys convexity: the feasible set is a scatter of points, not a polyhedron, and the problem is NP-hard in general. Dropping the integrality gives the **LP relaxation**, whose optimal value is a bound on the MILP optimum (a lower bound for a minimisation, an upper bound for a maximisation), because the relaxation optimises over a larger set. **Branch and bound** turns this bound into an exact method [10]:

1. Solve the LP relaxation of the current subproblem. If it is infeasible, or if its bound cannot beat the best integer solution found so far (the *incumbent*), discard the subproblem (*prune*).
2. If the relaxed solution is integral, it is the best solution of this subproblem; make it the incumbent if it is better than the current one.
3. Otherwise pick a variable z_j with fractional value v and *branch*: create two subproblems with the extra constraint $z_j \leq \lfloor v \rfloor$ and $z_j \geq \lceil v \rceil$ respectively, and repeat.

The method terminates because every branch cuts away a slice of the relaxed region that contains no integer point, and it is exact because a subproblem is pruned only when it provably

contains nothing better than the incumbent; its running time is exponential in the worst case but often small in practice, the trade-off that ?? explores.

Example A.30 (Branch and bound by hand). Maximise $5z_1 + 4z_2$ subject to $6z_1 + 4z_2 \leq 24$, $z_1 + 2z_2 \leq 6$, $z_1, z_2 \geq 0$ and integer. The relaxation (node 1) has the optimum $(3, 1.5)$ with value 21, so no integer solution can exceed 21. Branching on z_2 gives node 2 ($z_2 \leq 1$) with relaxed optimum $(3.33, 1)$ and bound 20.67, and node 3 ($z_2 \geq 2$) with $(2, 2)$ and bound 18. Branching node 2 on z_1 gives node 4 ($z_1 \leq 3$), whose relaxation $(3, 1)$ is integral with value 19 and becomes the incumbent, and node 5 ($z_1 \geq 4$), whose relaxation $(4, 0)$ is integral with value 20 and replaces it. Node 3 is pruned: its bound 18 cannot beat 20. The optimum is $(4, 0)$ with value 20, which enumerating all feasible integer points confirms; rounding the relaxed optimum to $(3, 2)$ would be infeasible and to $(3, 1)$ suboptimal.

Big- M constraints. Binaries make it possible to say “at least one of these constraints holds”, which no LP can express. Given a constant M larger than any value that $\mathbf{a}^\top \mathbf{z} - \beta$ can take on the feasible set, the pair

$$\mathbf{a}^\top \mathbf{z} \leq \beta + M b, \quad b \in \{0, 1\}, \quad (\text{A.34})$$

enforces the constraint when $b = 0$ and switches it off when $b = 1$. To keep a drone out of an axis-aligned box $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$, ?? writes the four “outside” conditions $x \leq x_{\min}$, $x \geq x_{\max}$, $y \leq y_{\min}$ and $y \geq y_{\max}$ in the form Equation (A.34) with binaries $b_1, \dots, b_4$ and adds $b_1 + b_2 + b_3 + b_4 \leq 3$, so that at least one condition stays enforced. The value of M matters: too small cuts off feasible positions, too large makes the LP relaxation so loose that branch and bound has little to prune, besides the numerical trouble of mixing coefficients of very different sizes.

A.4.6 Gradient descent

When a problem has no constraints and is too large, too nonlinear or too irregular for the solvers above, the tool of last resort is to follow the negative gradient:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k), \quad \alpha > 0 \text{ the step size (learning rate)}. \quad (\text{A.35})$$

By Proposition A.14, $f(\mathbf{x}_{k+1}) \approx f(\mathbf{x}_k) - \alpha \|\nabla f(\mathbf{x}_k)\|^2$, so a small enough step always decreases f ; the question is how small.

Proposition A.31 (Convergence of gradient descent). *Let f be convex and differentiable with an L -Lipschitz gradient, $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L \|\mathbf{x} - \mathbf{y}\|$, and let $\mathbf{x}^*$ be a minimiser. With a constant step $\alpha \leq 1/L$ the iterates of Equation (A.35) satisfy $f(\mathbf{x}_k) - f(\mathbf{x}^*) \leq \|\mathbf{x}_0 - \mathbf{x}^*\|^2 / (2\alpha k)$. If in addition f is μ -strongly convex, $\nabla^2 f \succeq \mu \mathbf{I}$, then $\|\mathbf{x}_k - \mathbf{x}^*\| \leq (1 - \alpha\mu)^k \|\mathbf{x}_0 - \mathbf{x}^*\|$ for $\alpha \leq 1/L$: the error shrinks geometrically, at a rate governed by the condition number L/μ .*

Proofs are in Boyd and Vandenberghe [2, Chapter 9] and Nocedal and Wright [6, Chapter 3]. The quadratic $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x}$ with $\mathbf{A} \succ \mathbf{0}$ shows the mechanism exactly: in the eigenbasis of $\mathbf{A}$ each coordinate is multiplied by $1 - \alpha\lambda_i$ per step, so the iteration converges iff $\alpha < 2/\lambda_{\max}$, and the slowest coordinate is the one with the smallest eigenvalue: a large eigenvalue ratio, a long narrow valley, forces a small step and a slow crawl along the valley floor.

Example A.32 (Step sizes on a narrow valley). Take $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 10x_2^2)$, so $\lambda_{\min} = 1$, $\lambda_{\max} = 10$ and $\nabla f = (x_1, 10x_2)^\top$, and start at $\mathbf{x}_0 = (9, 2)^\top$. With $\alpha = 0.05$ the factors are 0.95 and 0.5: the iterates drop to the valley floor and then crawl along it, reaching $\mathbf{x}_{20} = (3.23, 0.00)^\top$ with $f = 5.20$. With $\alpha = 0.18$ the factors are 0.82 and -0.80 : the iterates zigzag across the valley but make faster progress along it, $\mathbf{x}_{20} = (0.170, 0.023)^\top$ with

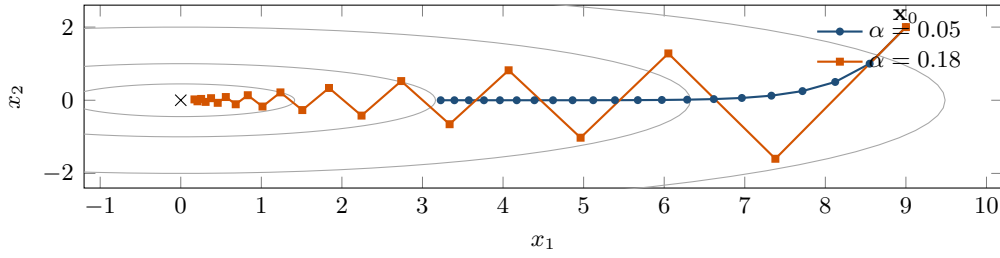


Figure A.4. Gradient descent on $f = \frac{1}{2}(x_1^2 + 10x_2^2)$ from $(9, 2)$ (Example A.32). The ellipses are level sets. The small step $\alpha = 0.05$ (blue) reaches the valley floor at once and then advances by only 5% per step; the larger step $\alpha = 0.18$ (orange) zigzags across the valley and gets much closer to the minimum in the same 20 steps. Any $\alpha > 0.2$ diverges.

$f = 0.017$. With $\alpha = 0.21$ the factor -1.1 on x_2 makes the iteration diverge: after 20 steps $x_2 = 13.5$ and $f = 905$. Figure A.4 shows the first two paths.

The same picture explains two phenomena in the book. The potential-field controller of ?? is gradient descent on its potential, with the drone's motion as the iteration: its oscillations in narrow corridors are the zigzag of Figure A.4, its local minima the non-convexity of Figure A.2. Training the prediction networks of ?? is gradient descent on a loss averaged over data, with the gradient obtained by the chain rule Equation (A.9) on random mini-batches (*stochastic gradient descent*); the learning rate is the α above, and momentum and adaptive step sizes are remedies for the narrow-valley effect.

A.5 Graphs and the Laplacian

A.5.1 Adjacency, degree and Laplacian matrices

An undirected graph $G = (V, E)$ with n vertices numbered $1, \dots, n$ is the communication network of ?? : the vertices are drones and an edge $\{i, j\}$ means that i and j can exchange messages. The neighbours of i are $N_i = \{j : \{i, j\} \in E\}$.

Definition A.33 (Adjacency, degree and Laplacian matrices). The **adjacency matrix** $\mathbf{A} \in \mathbb{R}^{n \times n}$ has $a_{ij} = a_{ji} = 1$ if $\{i, j\} \in E$ and 0 otherwise (for a weighted graph, $a_{ij} = w_{ij} \geq 0$). The **degree** of i is $d_i = \sum_j a_{ij}$, the degree matrix is $\mathbf{D} = \text{diag}(d_1, \dots, d_n)$, and the **graph Laplacian** is

$$\mathbf{L} = \mathbf{D} - \mathbf{A}, \quad (\mathbf{L}\mathbf{x})_i = \sum_{j \in N_i} a_{ij}(x_i - x_j). \quad (\text{A.36})$$

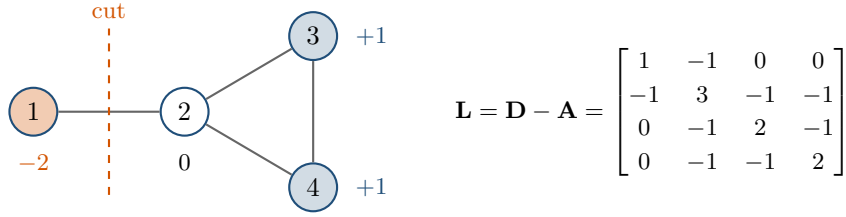
The second form of Equation (A.36) is the one to remember: applied to a vector of drone states, $\mathbf{L}$ returns for every drone the sum of its disagreements with its neighbours, which is exactly what a consensus protocol feeds back.

Proposition A.34 (The Laplacian quadratic form). For every $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{x}^\top \mathbf{L} \mathbf{x} = \sum_{\{i,j\} \in E} a_{ij} (x_i - x_j)^2$.

Proof. $\mathbf{x}^\top \mathbf{D} \mathbf{x} - \mathbf{x}^\top \mathbf{A} \mathbf{x} = \sum_i d_i x_i^2 - \sum_{i,j} a_{ij} x_i x_j = \frac{1}{2} \sum_{i,j} a_{ij} (x_i^2 + x_j^2 - 2x_i x_j) = \frac{1}{2} \sum_{i,j} a_{ij} (x_i - x_j)^2$, using $d_i = \sum_j a_{ij}$; each edge appears twice in the double sum, which removes the factor $\frac{1}{2}$. $\square$

Theorem A.35 (Spectrum of the Laplacian). Let $\mathbf{L}$ be the Laplacian of an undirected graph with eigenvalues $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ and maximum degree $d_{\max}$.

(i) $\mathbf{L}$ is symmetric and PSD.



degrees (1, 3, 2, 2), Fiedler vector $(-2, 0, 1, 1)$, eigenvalues $\lambda = (0, 1, 3, 4)$; $\lambda_2 = 1 > 0$: connected

Figure A.5. The network of Example A.36 and its Laplacian. Each diagonal entry is a degree and each -1 an edge. The entries of the Fiedler vector are written next to the vertices; the sign change across the edge $\{1, 2\}$ marks the cut that disconnects the graph.

- (ii) $\mathbf{L}\mathbf{1} = \mathbf{0}$, so $\lambda_1 = 0$ with eigenvector $\mathbf{1}$.
- (iii) The multiplicity of the eigenvalue 0 equals the number of connected components of G . In particular $\lambda_2 > 0$ iff G is connected.
- (iv) $\lambda_n \leq 2d_{\max}$.

Proof. (i) Symmetry is inherited from $\mathbf{A}$ and $\mathbf{D}$; $\mathbf{x}^T \mathbf{L} \mathbf{x} \geq 0$ by Proposition A.34. (ii) Every row of $\mathbf{L}$ sums to $d_i - \sum_j a_{ij} = 0$. (iii) By Proposition A.34, $\mathbf{x}^T \mathbf{L} \mathbf{x} = 0$ iff $x_i = x_j$ on every edge, that is, iff $\mathbf{x}$ is constant on each connected component; for a PSD matrix, $\mathbf{x}^T \mathbf{L} \mathbf{x} = 0$ iff $\mathbf{L} \mathbf{x} = \mathbf{0}$ (write $\mathbf{x}$ in the eigenbasis). The null space is therefore spanned by the indicator vectors of the components, which are linearly independent, so its dimension is the number of components, and this dimension is the multiplicity of 0 for a symmetric matrix. (iv) By Gershgorin's theorem every eigenvalue lies in one of the discs centred at $l_{ii} = d_i$ with radius $\sum_{j \neq i} |l_{ij}| = d_i$, hence in $[0, 2d_{\max}]$. $\square$

The eigenvalue λ_2 is the **algebraic connectivity** of the graph [5]: zero exactly when the network is cut, and the larger, the faster information spreads. ?? shows that under the consensus dynamics $\dot{\mathbf{x}} = -\mathbf{L} \mathbf{x}$ the average $\frac{1}{n} \mathbf{1}^T \mathbf{x}$ is conserved, because $\mathbf{1}^T \mathbf{L} = \mathbf{0}^T$ by (i) and (ii), and the disagreement decays like $e^{-\lambda_2 t}$ [7]. The discrete version $\mathbf{x}_{k+1} = (\mathbf{I} - \varepsilon \mathbf{L}) \mathbf{x}_k$ multiplies eigencomponent i by $1 - \varepsilon \lambda_i$ per step, so it converges iff $0 < \varepsilon < 2/\lambda_n$, and by (iv) any $\varepsilon < 1/d_{\max}$ is safe without knowing the spectrum.

Example A.36 (A four-drone network). The graph of Figure A.5 has the edges $\{1, 2\}$, $\{2, 3\}$, $\{3, 4\}$ and $\{2, 4\}$, so the degrees are (1, 3, 2, 2) and

$$\mathbf{L} = \begin{bmatrix} 1 & -1 & 0 & 0 \\ -1 & 3 & -1 & -1 \\ 0 & -1 & 2 & -1 \\ 0 & -1 & -1 & 2 \end{bmatrix}, \quad \lambda = (0, 1, 3, 4).$$

The graph is connected, so $\lambda_2 = 1 > 0$, and $\lambda_n = 4 \leq 2d_{\max} = 6$. The eigenvector of λ_2 , the Fiedler vector $(-2, 0, 1, 1)^T$, is negative on drone 1 and positive on drones 3 and 4: its sign pattern points at the bridge $\{1, 2\}$ as the weakest link. Deleting that edge isolates drone 1 and the spectrum becomes $(0, 0, 3, 3)$, with the double zero of Theorem A.35(iii). From $\mathbf{x}_0 = (4, 0, 1, -1)^T$ the consensus dynamics converge to the average 1 on every drone.

A.5.2 Shortest-path notation

The search chapters use a directed graph $G = (V, E)$ with a cost $c(u, v) \geq 0$ on every edge, successors $\text{Succ}(u) = \{v : (u, v) \in E\}$ and predecessors $\text{Pred}(u)$. A path $\pi = (v_0, \dots, v_m)$

follows edges and costs $\text{cost}(\pi) = \sum_i c(v_i, v_{i+1})$; the **shortest-path distance** $\text{dist}(u, v)$ is the least cost of a path from u to v , ∞ if there is none, and it satisfies the triangle inequality $\text{dist}(u, w) \leq \text{dist}(u, v) + \text{dist}(v, w)$. `??` computes $\text{dist}(s, \cdot)$ from a start s and writes it $\delta(s, v)$; `??` keeps for every node the cost $g(n)$ of the best path found so far, a heuristic estimate $h(n)$ of $\text{dist}(n, \gamma)$ to the goal γ and the priority $f = g + h$, with the frontier in OPEN and the expanded nodes in CLOSED. A 4- or 8-connected grid is such a graph with $|E| \leq 4|V|$ or $8|V|$, and the space-time graph of `????` has the $(T + 1)|V|$ states (v, t) , $t = 0, \dots, T$, with a wait edge from (v, t) to $(v, t + 1)$ beside the move edges.

A.6 Complexity and data structures

A.6.1 Asymptotic notation

Definition A.37 (Big-O, Omega and Theta). For functions $f, g: \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, $f(n) = \mathcal{O}(g(n))$ if there are constants $c > 0$ and n_0 with $f(n) \leq c g(n)$ for all $n \geq n_0$; $f(n) = \Omega(g(n))$ if $f(n) \geq c g(n)$ for all $n \geq n_0$; and $f(n) = \Theta(g(n))$ if both hold.

Big-O is an upper bound up to a constant factor for large inputs; Θ says the bound is tight. Constants and lower-order terms are dropped ($3n^2 + 50n = \Theta(n^2)$), the base of a logarithm is irrelevant, and the usual ladder is $1 \prec \log n \prec n \prec n \log n \prec n^2 \prec 2^n \prec n!$. A bound describes the worst case unless it says *expected* (average over random choices) or *amortised* (average over a sequence of operations), and it must be read with the right variables: Dijkstra's algorithm with a binary heap runs in $\mathcal{O}((|V| + |E|) \log |V|)$ (`??`), which on a grid with $|E| = \Theta(|V|)$ is $\mathcal{O}(|V| \log |V|)$; a space-time search over T steps has $(T + 1)|V|$ states; and the exponential worst case of the multi-agent solvers of `????` is in the number of agents, not in the size of the map.

A.6.2 Binary heaps

A **binary heap** stores n keys in an array $a[0], \dots, a[n - 1]$ read as a complete binary tree: the children of position i are $2i + 1$ and $2i + 2$, its parent is $\lfloor (i - 1)/2 \rfloor$, and the *heap property* says that no key is larger than its children's, so $a[0]$ is the minimum. Inserting appends the key and swaps it upwards while it is smaller than its parent (*sift-up*); extracting the minimum moves the last key to the root and swaps it downwards with its smaller child (*sift-down*); each walks one root-to-leaf path of length $\lfloor \log_2 n \rfloor$, hence $\mathcal{O}(\log n)$, and building a heap from an unsorted array by sifting down from the last internal node costs only $\mathcal{O}(n)$ [3, Chapter 6]. [Table A.3](#) collects the costs of the operations a search needs, with those of the Python containers used in the book's code.

A.6.3 Hashing, and the `heapq` and `dict` idiom

A **hash table** maps a key to an array position with a hash function and resolves the occasional collision by probing or chaining, so that insertion, lookup and deletion take constant expected time [3, Chapter 11]; `dict` and `set` are hash tables. Keys must be *hashable*, in Python immutable: tuples of integers such as a cell (x, y) or a space-time state (x, y, t) are ideal, lists and NumPy arrays are not accepted, and floating-point keys are legal but dangerous because two values equal on paper may differ in the last bit, which is why the search code of the book indexes cells by integer tuples.

The `heapq` module implements a binary heap on a plain list, but it has no decrease-key. The idiom of `????`, inherited by every planner built on them, is *lazy deletion*: keep the best known g of every node in a `dict`; when a better cost is found, push a fresh entry $(f, -g, \text{counter}, \text{node})$ and leave the old one in the heap; when an entry is popped, skip it if its node is already

Table A.3. Costs of the operations of a binary heap and of the Python containers used in the search code. Hash-table costs are expected costs; worst cases are $\mathcal{O}(n)$.

Structure	Operation	Cost	Python
binary heap	peek minimum	$\mathcal{O}(1)$	<code>heap[0]</code>
binary heap	insert	$\mathcal{O}(\log n)$	<code>heapq.heappush</code>
binary heap	extract minimum	$\mathcal{O}(\log n)$	<code>heapq.heappop</code>
binary heap	build from n items	$\mathcal{O}(n)$	<code>heapq.heapify</code>
binary heap	decrease-key	$\mathcal{O}(\log n)$	push again; skip stale entries
hash table	insert, lookup, delete	$\mathcal{O}(1)$ exp.	<code>dict</code> , <code>set</code>
dynamic array	append; index	$\mathcal{O}(1)$ amort.; $\mathcal{O}(1)$	<code>list.append</code> , <code>a[i]</code>
dynamic array	pop front; membership	$\mathcal{O}(n)$	<code>list.pop(0)</code> , <code>x in a</code>
sorted array	insert	$\mathcal{O}(n)$	<code>bisect.insort</code>
any	sort n items	$\mathcal{O}(n \log n)$	<code>sorted</code>

closed. The heap then holds at most one entry per relaxation, $\mathcal{O}(|E|)$ in total, which leaves the asymptotic bounds unchanged.

Heap entries must be comparable

`heapq` orders tuples lexicographically, so when two entries agree on f and g it compares the next field. If that field is a node without an ordering (a dictionary, an object) the search crashes at the first tie, and if it is a NumPy array it crashes at the first comparison. A running counter from `itertools.count()`, placed before the node, guarantees that the comparison never reaches the node and makes tie-breaking deterministic; the entry $-g$ before the counter prefers the larger g among equal f , as ?? recommends.

A.7 Summary

Chapter summary

- A symmetric matrix is a scaling along orthogonal axes, $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$; it is PSD iff all $\lambda_i \geq 0$, and every covariance is PSD. Block matrices are handled with the Schur complement, which gives the block inverse, the block determinant and the Woodbury identity.
- Least squares is solved by the normal equations $\mathbf{A}^\top \mathbf{A} \mathbf{x} = \mathbf{A}^\top \mathbf{b}$; never form an inverse to do it.
- Linearisation keeps the Jacobian and drops an error of second order in the displacement; the double-integrator step $\mathbf{p}' = \mathbf{p} + \mathbf{v}\Delta t + \frac{1}{2}\mathbf{a}\Delta t^2$, $\mathbf{v}' = \mathbf{v} + \mathbf{a}\Delta t$ is exact for a piecewise-constant input.
- Covariances propagate as $\mathbf{A}\mathbf{\Sigma}\mathbf{A}^\top$ under any linear map; Gaussians stay Gaussian under affine maps, marginalisation and conditioning, and the conditioning formula Equation (A.18) is the Kalman update in disguise.
- A filter is total probability (predict) followed by Bayes' rule (update); Monte Carlo errors shrink like $1/\sqrt{N}$; the squared Mahalanobis distance of a Gaussian vector is χ_n^2 , which sets gates such as 5.991 for $n = 2$ at 95 %.
- Convexity makes local minima global and the KKT conditions sufficient. An LP attains its optimum at a vertex, a convex QP is an LP with a quadratic cost, a MILP

is solved exactly by branch and bound with big- M binaries encoding disjunctions, and gradient descent converges for $\alpha < 2/\lambda_{\max}$ but crawls on ill-conditioned problems.

- The Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$ is PSD with $\mathbf{L}\mathbf{1} = \mathbf{0}$; $\lambda_2 > 0$ iff the graph is connected and sets the speed of consensus, and $\lambda_n \leq 2d_{\max}$ bounds the safe step size.
- A binary heap gives $\mathcal{O}(\log n)$ insertion and extraction; `heapq` plus a `dict`, lazy deletion and a tie-breaking counter is the priority queue of every search in this book.

Further reading. Linear algebra, convexity and optimality conditions are treated with great clarity in Boyd and Vandenberghe [2], whose appendices cover the matrix facts of [Appendix A.1](#); the algorithms of optimization are in Nocedal and Wright [6] and integer programming in Wolsey [10]. The probabilistic material follows Thrun, Burgard, and Fox [9] and Särkkä [8]; gating is treated in Bar-Shalom, Li, and Kirubarajan [1] and sequential Monte Carlo in Doucet, Freitas, and Gordon [4]. Spectral graph theory and consensus are surveyed in Olfati-Saber, Fax, and Murray [7], building on Fiedler [5]; heaps, hashing and asymptotic notation are in Cormen et al. [3].

Bibliography

- [1] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. Wiley, 2001.
- [2] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. 3rd ed. MIT Press, 2009.
- [4] A. Doucet, N. de Freitas, and N. Gordon, eds. *Sequential Monte Carlo Methods in Practice*. Springer, 2001.
- [5] M. Fiedler. “Algebraic Connectivity of Graphs”. In: *Czechoslovak Mathematical Journal* 23.2 (1973), pp. 298–305.
- [6] J. Nocedal and S. J. Wright. *Numerical Optimization*. 2nd ed. Springer, 2006.
- [7] R. Olfati-Saber, J. A. Fax, and R. M. Murray. “Consensus and Cooperation in Networked Multi-Agent Systems”. In: *Proceedings of the IEEE* 95.1 (2007), pp. 215–233.
- [8] S. Särkkä. *Bayesian Filtering and Smoothing*. Cambridge University Press, 2013.
- [9] S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. MIT Press, 2005.
- [10] L. A. Wolsey. *Integer Programming*. Wiley, 1998.

Index

adjacency matrix, 25
algebraic connectivity, 26

Bayes' rule, 19
big-M constraint, 24
big-O notation, 27
binary heap, 27
branch and bound, 23

Cauchy–Schwarz inequality, 11
chain rule, 14
chi-square distribution, 20
Cholesky factorisation, 12
convexity
 function, 20
 set, 20
covariance, 16
cross product, 11

dot product, 11
double integrator, 16

eigenvalue, 11
expectation, 16

gating, 20
Gaussian
 conditional, 17
 marginal, 17
 multivariate, 17
gradient, 14
gradient descent, 24
graph Laplacian, 25

hash table, 27
Hessian, 14

importance sampling, 20

Jacobian, 14

KKT conditions, 21

Lagrangian, 21
lazy deletion, 27
least squares, 14
linear program, 22
 standard form, 22
linearisation, 15
LP relaxation, 23

Mahalanobis distance, 17
mathematical refresher, 10–29
matrix

 inverse, 11
 positive definite, 12
 positive semidefinite, 12
 rank, 11
 transpose, 11

mixed-integer linear program, 23
Monte Carlo estimation, 19

norm
 1-norm and infinity-norm, 11
 Euclidean, 11
normal equations, 14

quadratic form, 12
quadratic program, 22

random vector, 16

Schur complement, 13
shortest-path distance, 27

total probability, 19